You've Got a Golden Ticket: Improving Generative Robot Policies With A Single Noise Vector

Omkar Patil[1,2] **Ondrej Biza**[1] **Thomas Weng**[1] **Karl Schmeckpeper**[1]
Wil Thomason[1] **Xiaohan Zhang**[1] **Kausik Sivakumar**[1] **Robin Walters**[1,3]
Nakul Gopalan[2] **Sebastian Castro**[1] **Stephen Hart**[1] **Eric Rosen**[1]

[1]Robotics and AI Institute (RAI) [2]Arizona State University [3]Northeastern University

Abstract: What happens when a pretrained generative robot policy is provided a constant initial noise as input, rather than repeatedly sampling it from a Gaussian? We demonstrate that the performance of a pretrained, frozen diffusion or flow matching policy can be improved with respect to a downstream reward by swapping the sampling of initial noise from the prior distribution (typically isotropic Gaussian) with a well-chosen, constant initial noise input—a *golden ticket*. We propose simple search methods to find golden tickets using Monte-Carlo policy evaluation that keeps the pretrained policy frozen, does not train any new networks, and is applicable to all diffusion/flow matching policies (and therefore many VLAs). Our approach to policy improvement makes no assumptions beyond being able to inject initial noise into the policy and calculate (sparse) task rewards of episode rollouts, making it deployable with no additional infrastructure or models. Our method improves the performance of policies in 46 out of 51 tasks across simulated and real-world robot manipulation benchmarks, with absolute improvements in success rate by up to 55% for some simulated tasks, and 28% within 60 search episodes for real-world tasks. Our approach naturally extends to multi-task settings, where we improve the average performance of a VLA policy across 7 tasks by 14% using a single noise vector. Further, we find that, a golden ticket optimized for one task can also boost performance in other related tasks for the same VLA policy. We release a codebase with pretrained policies and golden tickets for simulation benchmarks using VLAs, diffusion policies, and flow matching policies: https://lottery-tickets.rai-inst.com/.

Keywords: Lottery Tickets, Policy Improvement, Generative Policies

1 Introduction

Conditional diffusion [1] and flow matching models [2] are popular approaches for learning robot control policies. Their ability to represent high-dimensional, multimodal action distributions have made them popular in both single-task policies and large-scale multi-task Vision-Language-Action (VLA) models. However, improving their out-of-the-box performance on downstream tasks currently introduces many computational and model design challenges. Existing policy improvement methods involve either 1) updating the pretrained model weights [3] (which is computationally expensive for large models like VLAs), 2) training an additional network [4, 5] (requiring complex model design choices), or 3) assuming access to external critic networks [6] or specific training regimes [7] (limiting the scenarios in which they can be applied).

Instead, we propose a new method that 1) keeps the pretrained policy weights frozen, 2) does not train an additional network, and 3) can be applied to any diffusion or flow matching framework without additional training or test-time assumptions. Our approach is motivated by a simple question:

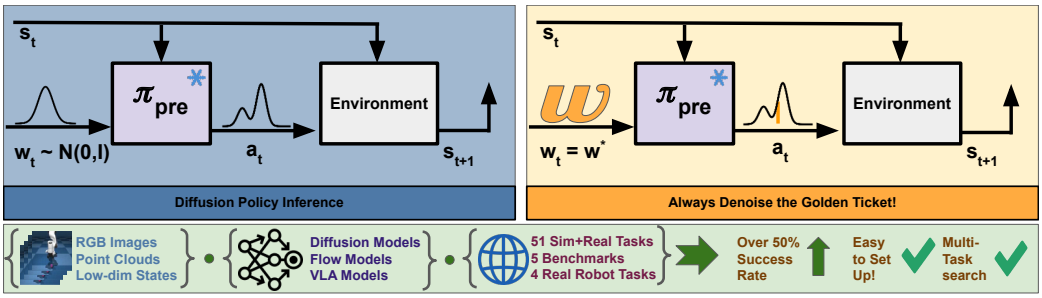


Figure 1: Overview of standard diffusion policy inference (left) versus our proposed inference approach of using golden tickets (right). Given a frozen, pretrained diffusion or flow matching policy π_{pre}, rather than sampling from a Gaussian every time an action needs to be computed, we use a constant, well-chosen initial noise vector w, called a golden ticket. We find golden tickets improve policy performance across a range of observation inputs, model architectures, and embodiments.

What happens when a pretrained generative robot policy is provided a constant initial noise as input, rather than repeatedly sampling from a Gaussian?

In this work, we demonstrate that the performance of a pretrained, frozen diffusion or flow matching policy can be improved by simply replacing the sampling of initial noise from the prior distribution with a well-chosen, constant initial noise input, which we call a *golden ticket* (Figure 1). Given a pretrained policy, a reward function, and an environment for a new downstream task, we pose finding golden tickets as a search problem, where candidate initial noise vectors, which we call *lottery tickets*, are optimized to find the ticket that maximizes cumulative discounted expected rewards on the downstream task. We contextualize golden ticket search in light of DSRL [5], a reinforcement learning approach to noise optimization, and show that a single fixed noise vector recovers most of its gains without any gradient updates. Beyond this, golden tickets unlock multi-task latent steering for policy improvement: a single ticket can boost performance across several tasks when searched jointly, and tickets optimized for one task may transfer to unseen tasks within the same suite.

Our experiments show an improvement over base policy performance in 46 out of 51 tasks across five simulated manipulation benchmarks and 4 real-world tasks with a variety of pretrained policy classes. When we apply our approach on hardware, we increase the relative success rate of an object picking task by 18%, and a piston assembly task by 28%, with the ticket search taking less than 150 episodes per task. Our contributions are:

1. A novel lottery ticket hypothesis for improving pretrained robot policies by using a single initial noise vector—a *golden ticket*—to optimize downstream task rewards.
2. Demonstration of the existence, prevalence and empirical performance of golden tickets across model architectures, observation modalities, and embodiments in 51 real and simulated tasks.
3. Search algorithms for finding golden tickets via Monte-Carlo policy evaluation, and experimental comparisons against state-of-the-art latent-steering and gradient-based methods.
4. Evidence that golden tickets extend to multi-task VLA settings: a ticket searched on one task may transfer to other tasks in the same suite (30 related tasks across three LIBERO suites using SmolVLA), and joint multi-task search yields tickets that lifts the multi-task average on SimplerEnv (WidowX) (GR00T N1.5).
5. Open-source code with golden tickets (and ticket search implementations) for pretrained VLA, diffusion, and flow-matching policies in three simulated manipulation benchmarks.

2 Background

Markov Decision Processes. We model robot manipulation as a Markov Decision Process (MDP). An MDP comprises a tuple $\{S, A, T, R, p_0, \gamma\}$, where S and A are sets of states and actions (respectively), $T(s' \mid s, a) = \Pr(s' \mid s, a)$ is the transition dynamics, $R(s, a)$ is the reward function,

$p_0 \in \triangle_S$ is the initial state distribution over S, and γ is the discount factor. A policy π rolls out episodes from an initial state $s_0 \sim p_0$ by iteratively sampling an action $a_t = \pi(*|s_t)$ to execute at each time step t to get the next state $s_{t+1} \sim T(*|s_t, a_t)$ and reward $r_t = R(s_t, a_t)$. The Q-function $Q^\pi(s, a)$ for π gives the cumulative discounted expected rewards of π rolled out from a state s with initial action a, following π for all subsequent actions. That is, $Q^\pi(s, a) = \mathbb{E}_{P^{\pi,T}(*)}[\sum_{t>0} \gamma^t R(s_t, a_t) \mid s_0 = s, a_0 = a]$.

Diffusion and Flow Matching. These generative methods iteratively refine a sample from a source distribution into a desired target data distribution. We focus on the flow matching framework with a Gaussian source distribution for illustration. In flow matching, given a dataset $D = \{s_t, a_t \sim p_{\text{data}}\}_{t=1}^n$, the forward process gradually corrupts the action $a \in D$ over time τ[1] by linearly interpolating it with a sample from an isotropic Gaussian: $a^{(\tau)} = (1 - \tau)\, a^{(0)} + \tau w$, where $w \sim \mathcal{N}(0, I)$ and $a^{(\tau)}$ represents the partially noised action $a^{(0)}$ at time τ. Generating new samples from the data distribution p_{data} requires reversing the forward process. Specifically, a denoising model $\hat{u} = \hat{u}(s, a^{(\tau)}, \tau)$ is trained to undo the forward process, and new samples from p_{data} can be generated by iteratively denoising a sample $a^{(\tau)}$ with $a^{(1)} \sim \mathcal{N}(0, I)$, treating the denoising model $\hat{u}$ as a velocity field and solving the ODE $\hat{a}^{(0)} = a^{(1)} + \int_1^0 \hat{u}(s, a^{(\tau)}, \tau)\, d\tau$. While diffusion and other flow variants differ in how they define the forward and reverse processes, our method is agnostic to these variations; it focuses only on making $a^{(1)}$, the initial noise vector used for inference, a constant vector rather than repeatedly sampling it from a Gaussian distribution. We consider DDIM sampling for diffusion models due to more efficient generation [8], which is critical in robotics.

3 Related Work

We review the lottery ticket hypotheses on optimizing initial noise vectors to improve performance of diffusion models, as well as existing approaches to policy improvement via latent steering.

Lottery Ticket Hypothesis for Diffusion/Flow Models. Our work is motivated by a "lottery ticket hypothesis" proposed by Mao et al. [9] in the context of image generation[2]: "randomly initialized Gaussian noise images contain special pixel blocks (winning tickets) that naturally tend to be denoised into specific content independently." Followup works have investigated how these "winning tickets" (also referred to as "golden noises" [11, 12, 13], or "golden tickets" in our work) can be optimized to achieve higher text-image alignment and higher human preference when they are used instead of sampling from isotropic Gaussian noise. Prior works have used a variety of metrics, such as noise inversion [14] and stability [15], semantic and natural appearance consistency [16], 3D geometry [17], and regularization to the original Gaussian distribution [18]. However, finding golden tickets for robot control policies present additional difficulties due to the temporal nature of the problem and dimensionality of the problem space (see Appendix A.1).

Policy Improvement via Latent Steering. While there is a diverse range of policy improvement methods, we focus on those that use latent steering, since they most closely relate to our approach. Diffusion Steering via Reinforcement Learning (DSRL) [5] swaps the source noise distribution with an observation-conditioned noise policy trained via reinforcement learning (RL). DSRL freezes the weights of a pretrained policy, providing strong regularization towards the behaviors already encoded in the pretrained model, and can be applied to any diffusion or flow matching policy. However, DSRL presents two major challenges in practice: 1) it trains an additional neural network, which involves modeling decisions to ensure sufficient model capacity, and 2) it integrates with an RL framework to train the noise policy, which requires hyperparameter tuning [5]. We instead pursue the paradigm of golden noises, or tickets, in which we *search* for a single initial noise vector that maximizes task rewards across environment states. Because golden tickets are akin to a noise policy that is observation-independent (unlike DSRL), they (1) require no additional trainable network

[1] We use a superscript (τ) for the forward/reverse process time to distinguish it from the MDP time subscript t introduced in the background on MDPs.

[2] The term "lottery ticket hypothesis" was first introduced by Frankle and Carbin [10] in the context of finding sparse subnetworks within denser networks, though this approach is not directly related to our work.

beyond the original policy, and (2) naturally extend to multi-task settings for policy improvement. A broader treatment of policy improvement methods can be found in Appendix A.2.

4 Lottery Ticket Hypothesis for Robot Control

Our main contribution is framing the search for a golden ticket, a fixed initial noise vector, as a mechanism for improving generative policy performance on a downstream task. We assume we are given a pretrained, frozen diffusion or flow matching robot control policy π, and treat this policy as a black box without access to any intermediate steps of the denoising process. Given an MDP representing a downstream task from which we can sample episode transitions, our goal is to adapt the policy π's behavior to maximize rewards while keeping the parameters frozen. We propose:

The Lottery Ticket Hypothesis for Robot Control. For a pretrained diffusion or flow matching policy and a downstream task, there exists a fixed initial noise vector w^* that, used in place of sampling from the Gaussian prior, increases the policy's expected return on held-out task evaluations.[3]

We call such a noise vector a *golden ticket* and test the hypothesis empirically in Section 5.2.1. Searching for a golden ticket on a robot policy raises several challenges: (1) the policy acts in an environment where current actions affect future states, (2) collecting samples from that environment is costly, and (3) the reward function may not be differentiable. In this work, we develop a search-based method using Monte-Carlo policy evaluation to address these challenges.

Search for Golden Tickets. We propose to *search* for golden tickets w^* with Monte-Carlo estimates of episodic-reward obtained from policy rollouts as the objective. Formally, let $\pi_w(s)$ denote a policy whose denoising process is always initialized with noise $w \in \mathbb{R}^{H \times d_a}$, where H is the horizon and d_a is the per-step action dimension; the base policy $\pi(s)$ samples a fresh $w \sim \mathcal{N}(0, I)$ at every action step. We seek w^* that maximizes the cumulative discounted reward obtained by π_w, averaged over M rollouts from fixed initial states $\{s_0^{(m)}\}_{m=1}^{M}$: that is, $w^* = \text{argmax}_w \frac{1}{M} \sum_{m=1}^{M} \sum_{t>0} \gamma^t R(s_t^{(m)}, \pi_w(s_t^{(m)}))$. In a multi-task setting, the objective is averaged across tasks as well, with the same ticket w used for all tasks.

In this work, we consider random search (RS) [19], zeroth-order search (ZOS) [19] and cross-entropy method (CEM) [20] for finding golden tickets. The simplest of them, random search samples N tickets, and returns one with the maximum average reward. We describe the algorithmic details for RS, ZOS and CEM, along with sequential halving [21] in Appendix B. Our approach only assumes that initial noise can be injected into the pretrained policy, and that (sparse) task rewards can be calculated for rollouts. This approach has many appreciable benefits: 1) it does not require setting up any additional infrastructure or training models, and 2) it is applicable to all diffusion and flow matching policies, that can be run as black-box systems, including many VLAs.

Evaluating Golden Tickets. To determine if the returned ticket is "golden" (i.e., that it has higher average performance than sampling from the Gaussian prior), we assume there is a held-out set of initial states or environments which we use to evaluate the performance of both the best ticket and the base policy. For a fixed compute budget, there exists a trade-off between number of tickets N and number of search environments M on which to evaluate those tickets. More tickets result in better coverage and potentially better search performance, but risk not generalizing to held-out environments. Conversely, more search environments improve the chance a ticket's performance transfers to held-out environments, but reduce overall performance by leaving fewer tickets evaluated.

5 Experiments

Our experimental evaluation is designed to assess the lottery ticket hypothesis for robot control policies and provide empirical evidence of its effectiveness and practical utility.

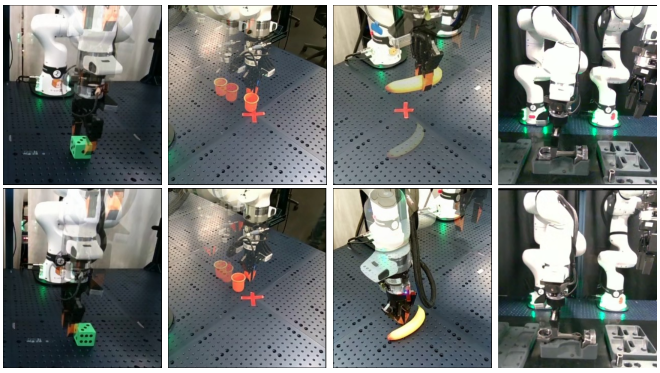
[3] Empirically, this holds for 46 of the 51 tasks in our benchmarks for a fixed number of trials (Appendix D.2).

5.1 Experimental Benchmarks

We investigate a wide range of model architectures, observation inputs, and environments. On real Franka hardware (Figure 2), we evaluate two RGB diffusion policies (piston assembly and cube picking) and two point-cloud diffusion policies (banana picking and cup pushing); all hardware experiments optimize a binary end-of-episode success signal. We also report results on the GR00T N1.5 [22] public checkpoint for 7 tasks of the **SimplerEnv** (WidowX) [23] environment, designed to reflect real world performance. Additionally, we consider 4 simulation benchmarks:

- **franka_sim** [24]: cube picking, flow-matching policy on low-dim state.
- **LIBERO** [25]: 30 tasks (OBJECT/GOAL/SPATIAL) with the public SmolVLA [26] checkpoint.
- **robomimic** [27]: 4 tasks with pretrained diffusion policies from DPPO [3] on low-dim state.
- **DexMimicGen** [28]: 5 bimanual tasks with RGB diffusion policies we train via BC.

In total, the suite covers 51 tasks and spans flow matching, BC-trained diffusion and VLA policies, with state, RGB, point-cloud, and language inputs across both simulation and real hardware; see Appendix C for full per-benchmark details.



(a) Pick cube (b) Push cup (c) Pick banana (d) Piston

Figure 2: Rollouts from diffusion policies sampling with Gaussian noise that we use in our hardware experiments. (top) An example successful rollout; (bottom) An example failed rollout.

5.2 Experimental Results

5.2.1 Do golden tickets for robot control policies exist, and how prevalent are they? How much do they improve performance compared to sampling Gaussian noise?

We evaluate the existence and prevalence of golden tickets across the 51 task–policy combinations in our suite. We run a large-scale search for golden tickets, with search budgets ranging from less than 250 tickets with 5 search environments each in SimplerEnv to 5000 tickets per task with 100 search environments each in robomimic (see Appendix D.1 for more details). We find that *golden tickets outperform Gaussian noise in* 46 *out of* 51 *tasks, and at least match it in* 49 (Figure 3). Under the null hypothesis that a searched noise vector outperforms Gaussian sampling no more often than chance ($H_0 : p \leq 0.5$), the observed proportion is highly significant: one-sided binomial p-value $\approx 1.2 \times 10^{-9}$, with Wilson 95% CI for p of $[0.79, 0.96]$ (Appendix D.2). In franka_sim cube picking, the base policy averages 38.5% success and the best golden tickets average 96%. In robomimic, golden tickets are found in 3 of 4 tasks (not `Square`), with the most extreme gain on `Can`, from 42.8% to 80.8%. In LIBERO, evaluating per-task golden tickets (Table 1), we obtain +13%, +12.8%, and +8% improvements over the base policy on LIBERO-SPATIAL, LIBERO-GOAL, and LIBERO-OBJECT, respectively. In DexMimicGen, golden tickets are found in 4 of 5 tasks; the biggest improvement is on `Box Cleanup` (87.6%→97.8%), while `Threading` shows base 62% vs. best ticket 60%. In SimplerEnv (WidowX), the average improvement is +26.5% with the largest gain on `put_eggplant_in_sink` (21% → 76.3%). On real Franka hardware, the

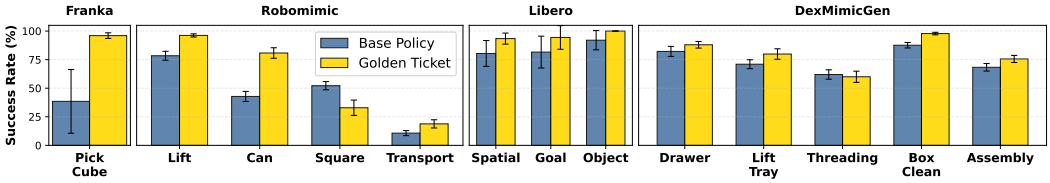
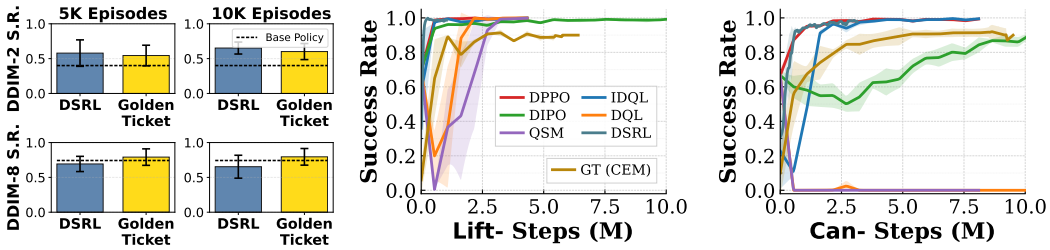


Figure 3: Comparison of task performance of the base policy (**blue**, left) and our approach using golden tickets (**gold**, right) on simulated benchmarks found using Random Search. We report mean and standard deviation of success rates (details in Section 5.2). Our open-source repository contains code and golden tickets for the first three benchmarks: franka_sim, robomimic, and LIBERO.



(a) DSRL vs. RS (b) robomimic `Lift` (c) robomimic `Can`

Figure 4: Comparison of our method, Golden Tickets (GT), a gradient-free policy improvement method with state-of-the-art latent-steering and gradient-based RL baselines. **(a)** DSRL vs. RS averaged across all 5 DexMimicGen tasks, across DDIM steps (2 vs. 8) and episode budgets (5k vs. 10k). **(b, c)** Eval success rate vs. environment steps, comparing GT (CEM) against RL baselines.

average improvement is +31%. Beyond average improvement, we examine when golden tickets reach a high *absolute* success rate: among the 34 task–policy pairs with base success rate $\geq 71\%$, golden tickets achieve $\geq 90\%$ success in 31 of them; among the 15 pairs with base $\geq 87.6\%$, they achieve $\geq 97\%$ in 14 of them (Appendix D.3). This suggests that golden tickets present a meaningful path to perfecting the performance of good behavior cloning policies. Full per-benchmark details and search budgets are in Appendix D.

5.2.2 How competitive is searching for golden tickets to state-of-the-art latent steering and reinforcement learning methods?

We compare CEM (Algorithm 2) against six gradient-based RL baselines including DSRL with tuned implementations for two state-based robomimic tasks, `Lift` and `Can` (Figure 4b,c). The other five gradient-based baselines either adapt the pretrained policy behavior (DPPO[3], IDQL[29], DQL[30]), or use diffusion but learn from scratch (DIPO[31], QSM[32]); see Ren et al. [3] for baseline details. On robomimic, merely searching for a good fixed initial noise is competitive with gradient-based adaptation methods, even for MLP diffusion policies trained in state-based tasks – outperforming QSM, DQL, and IDQL on `Can`. Notably, a single noise vector is able to recover most of the gains of DSRL on these two tasks. In higher-dimensional visual tasks (Figure 4a), we outperform DSRL on `BoxCleanup` across DDIM steps even with random search, while DSRL tends to perform better with 2 DDIM steps on other tasks (Appendix D.7). Overall, the results suggest that searching for golden tickets can be a simple and effective policy improvement tool. A comparison of RS, CEM, and ZOS is in Appendix D.6, along with search ablations and results on bandit-strategies for sample-efficiency. We generally find CEM to be the most effective at finding golden tickets, while ZOS collapses for certain seeds.

LIBERO-Goal											
#Ticket	**T0**	**T1**	**T2**	**T3**	**T4**	**T5**	**T6**	**T7**	**T8**	**T9**	**Avg**
Base	72	94	86	52	92	78	80	**100**	94	68	**81.6**
#03c2 ⋆	84	**100**	92	40	**100**	64	**94**	**100**	98	18	79.0
#2cee	78	84	**100**	2	98	76	22	**100**	86	0	64.6
#1ff8	98	64	86	14	**100**	0	38	96	**100**	2	59.8
#4d97	2	96	16	**68**	98	92	30	54	**100**	4	56.0
#672f	0	12	88	0	72	4	38	**100**	2	**90**	40.6
#0f3f	**100**	2	0	0	68	56	0	76	38	0	34.0
#0310	0	12	0	0	**100**	**100**	10	4	42	0	26.8

LIBERO-Object											
#Ticket	**T0**	**T1**	**T2**	**T3**	**T4**	**T5**	**T6**	**T7**	**T8**	**T9**	**Avg**
Base	82	98	98	98	76	82	**100**	90	96	**100**	**92.0**
#015a ⋆	36	62	**100**	**100**	16	92	**100**	94	**100**	**100**	80.0
#5159	20	86	**100**	**100**	60	**100**	94	36	92	**100**	78.8
#184f	98	0	94	86	**100**	0	78	**100**	86	44	68.6
#1944	**100**	38	72	94	94	4	30	90	72	0	59.4
#14e7	14	**100**	0	0	0	32	56	64	56	0	32.2
#096d	50	0	0	4	0	0	0	**100**	0	0	15.4

LIBERO-Spatial											
#Ticket	**T0**	**T1**	**T2**	**T3**	**T4**	**T5**	**T6**	**T7**	**T8**	**T9**	**Avg**
Base	78	94	84	62	84	58	90	90	78	86	**80.4**
#60c0 ⋆	82	82	88	**100**	74	24	98	80	82	68	77.8
#a68f	76	72	92	50	**92**	**90**	56	70	**90**	82	77.0
#ecf0	68	96	80	76	12	30	98	**96**	82	68	70.6
#0e0c	80	60	86	**100**	10	24	90	92	84	74	70.0
#05c7	70	92	88	72	52	0	**100**	64	44	78	66.0
#517b	72	84	**92**	78	2	20	66	92	62	52	62.0
#6a21	64	**98**	90	30	**92**	30	30	74	58	44	61.0
#7c0e	64	68	64	4	22	60	6	88	44	**92**	51.2
#3b9d	**84**	36	46	94	14	6	88	22	16	0	40.6

Table 1: Success rates (%) for base policy and lottery tickets on LIBERO-GOAL, LIBERO-OBJECT, and LIBERO-SPATIAL with SmolVLA. T0 - T9 represent different tasks. Top-1 ticket per task shown (unique set) plus best average ticket (⋆). Underline: $\geq$ base policy performance.

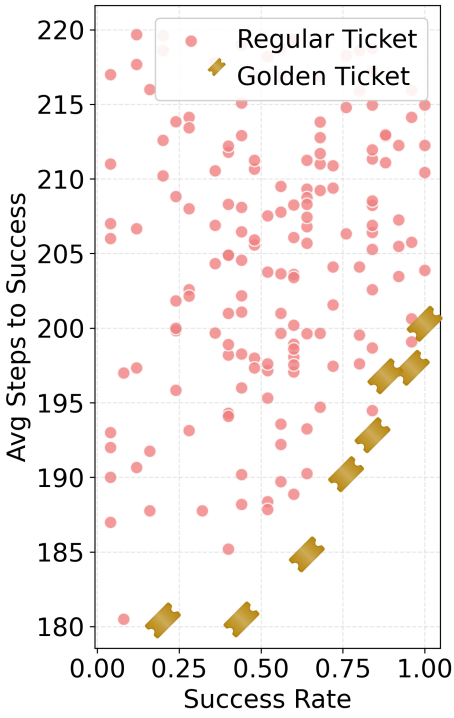


Figure 5: Various tickets (pink) for the franka_sim `pick` policy, evaluated according to success rate and speed (determined by length of successful episodes). Right is better success rate, lower is faster time to success. Because lottery tickets exhibit extreme differences in policy performance, a Pareto frontier is defined by tickets that are further right/down than others (represented with golden ticket icons).

5.2.3 Do golden tickets optimized for one task act as golden tickets for other tasks? Can we do multi-task policy improvement using golden tickets?

We address multi-task policy improvement from two perspectives on two different benchmarks. In LIBERO with SmolVLA across the 3 task suites (LIBERO-OBJECT, LIBERO-GOAL, LIBERO-SPATIAL; 10 tasks each, 30 tasks total), we investigate whether golden tickets found (using random search) for one task can also act as golden tickets for other tasks. Across all 30 tasks, golden tickets match or exceed the base policy's performance (Table 1). Certain golden tickets match or exceed the base policy on multiple tasks: in LIBERO-GOAL, ticket `#03c2` matches or exceeds the base policy in $7/10$ tasks (`T0`, `T1`, `T2`, `T4`, `T6`, `T7`, `T8`); in LIBERO-OBJECT, ticket `#015a` matches or exceeds the base policy in $7/10$ tasks (`T2`, `T3`, `T5`, `T6`, `T7`, `T8`, `T9`); in LIBERO-SPATIAL, ticket `#60c0` matches or exceeds the base policy in $5/10$ tasks (`T0`, `T2`, `T3`, `T6`, `T8`). Our results provide evidence of multi-task golden tickets and that related tasks may share golden tickets for VLA policies.

On SimplerEnv (WidowX), we run multi-task golden ticket search with the task-averaged return as the objective. For `put_eggplant_in_sink` and `put_eggplant_in_basket`, golden tickets outperform the base on both tasks by over 30% average. With a search budget of 500 tickets and 5 evaluations per task, we find multi-task golden tickets that beat the base policy by over 14% on

average over 7 tasks, while regressing on only $2/7$ of them (full results in Appendix D.8). Our results demonstrate that golden tickets naturally extend to the multi-task regime.

5.2.4 Do lottery tickets define a Pareto frontier when given multiple objectives to optimize?

We investigate this in franka_sim with a block picking task under two objectives—maximizing success rate, and maximizing speed of successful episodes—which trade off against each other (the faster the robot moves, the harder it becomes to successfully pick up the block). We run 400 tickets in 50 different start states each, and evaluate each ticket on both objectives. *Golden tickets are sufficiently varied to define a Pareto frontier that balances success rate and speed for an object picking task (Figure 5).* Ticket success rates vary from $\approx 5\%$ to nearly 100%, and speeds of successful episodes range from ≈ 180 to 220 steps; this diversity results in a small subset of golden tickets that define a Pareto frontier, outperforming all other regular tickets to the right and below them and representing a balance between the two objectives. This suggests a simple way to alter a robot's behavior to adapt between objectives online: collect the golden tickets on the Pareto frontier, and switch between them as desired.

5.2.5 How effective are golden tickets in practice and how do they perform on hardware?

On hardware, golden ticket search delivers substantial gains within 1 hour of rollout time on held-out episodes for every task:

- `block pick`: $80\% \rightarrow 98\%$ ($+18\%$) on 50 held-out episodes; 6 tickets $\times 25$ episodes $= 150$ search episodes ($\approx$30 min). The same search also surfaces a ticket succeeding only $2/50$ times, letting us steer the policy to *miss* with extreme reliability.
- `piston assembly`: $60\% \rightarrow 88\%$ ($+28\%$) on 25 eps; 22 tickets over 61 search eps ($\approx$45 mins).
- `banana pick`: $50\% \rightarrow 68\%$ ($+18\%$) on 50 eps; 10 tickets $\times 5$ eps $= 50$ search eps ($\approx$11 min).
- `cup push`: $40\% \rightarrow 100\%$ ($+60\%$) on 10 eps; 10 tickets $\times 5$ eps $= 50$ search eps ($\approx$10 min).

In line with showing improvements on public checkpoints, we also search for golden tickets on the GR00T N1.5 checkpoint for the SimplerEnv (WidowX), an environment designed to correlate closely with real world performance. Across 3 seed trials for each task individually, we are able to improve the base VLA performance by an average of 20% (with 5 of 7 showing substantial gains) using a search budget of less than 100 tickets on average. We use 5 evaluations per ticket for 4 tasks and 10 evaluations per ticket for 3 tasks with relatively higher performance. Average single-task gains reach $+48\%$ on `put_eggplant_in_sink` ($21\% \rightarrow 69.3\%$), $+34\%$ on `close_drawer` ($65\% \rightarrow 99.2\%$), and $+28\%$ on `put_eggplant_in_basket` ($63\% \rightarrow 91.6\%$). Full results can be found in Appendix D.4.

6 Limitations and Future Work

There are important limitations to golden tickets that are addressable in future work. While we do not make any assumptions about the pretrained policy, our method only works if the policy is steerable [5], which is not guaranteed. While golden tickets make DDIM inference deterministic, we show in Appendix D.6 that top-k sampling among golden tickets recovers stochastic behavior at no cost to performance; a fuller characterization of multimodal action coverage is left to future work. Third, reaching perfect performance may demand stronger search methods than ours, particularly when the base policy itself is weak; designing such methods is a direction for future work.

7 Conclusion

In this work, we propose an approach to improving pretrained diffusion or flow matching policies that avoids 1) adjusting the original policy weights, 2) training additional neural networks, and 3) making training or test time assumptions on the base model. Our method is based on a proposed

lottery ticket hypothesis for robot control, that the performance of a pretrained, frozen diffusion or flow matching policy can be improved by swapping the sampling of initial noise from the prior distribution (typically an isotropic Gaussian) with a well-chosen, constant initial noise input, called a golden ticket. Through our experiments, we demonstrate that golden tickets 1) often exist and outperform using Gaussian noise, 2) can be competitive with state-of-the-art latent steering methods trained with RL, and 3) naturally extend to multi-task policy improvement. We also release an open-source codebase with models, environments and golden tickets for VLAs, diffusion policies, and flow matching policies.

8 Acknowledgements

We thank Stefanie Tellex for providing valuable feedback on the manuscript.

References

[1] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models, 2020. URL https://arxiv.org/abs/2006.11239.

[2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling, 2023. URL https://arxiv.org/abs/2210.02747.

[3] A. Z. Ren, J. Lidard, L. L. Ankile, A. Simeonov, P. Agrawal, A. Majumdar, B. Burchfiel, H. Dai, and M. Simchowitz. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024.

[4] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018.

[5] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine. Steering your diffusion policy with latent space reinforcement learning. *arXiv preprint arXiv:2506.15799*, 2025.

[6] Y. Wang, L. Wang, Y. Du, B. Sundaralingam, X. Yang, Y.-W. Chao, C. Perez-D'Arpino, D. Fox, and J. Shah. Inference-time policy steering through human interactions, 2025. URL https://arxiv.org/abs/2411.16627.

[7] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, D. Driess, M. Equi, A. Esmail, Y. Fang, C. Finn, C. Glossop, T. Godden, I. Goryachev, L. Groom, H. Hancock, K. Hausman, G. Hussein, B. Ichter, S. Jakubczak, R. Jen, T. Jones, B. Katz, L. Ke, C. Kuchi, M. Lamb, D. LeBlanc, S. Levine, A. Li-Bell, Y. Lu, V. Mano, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, C. Sharma, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, W. Stoeckle, A. Swerdlow, J. Tanner, M. Torne, Q. Vuong, A. Walling, H. Wang, B. Williams, S. Yoo, L. Yu, U. Zhilinsky, and Z. Zhou. $\pi^*_{0.6}$: a vla that learns from experience, 2025. URL https://arxiv.org/abs/2511.14759.

[8] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models, 2022. URL https://arxiv.org/abs/2010.02502.

[9] J. Mao, X. Wang, and K. Aizawa. The lottery ticket hypothesis in denoising: Towards semantic-driven initialization. In *European Conference on Computer Vision*, pages 93–109. Springer, 2024.

[10] J. Frankle and M. Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks, 2019. URL https://arxiv.org/abs/1803.03635.

[11] Z. Zhou, S. Shao, L. Bai, S. Zhang, Z. Xu, B. Han, and Z. Xie. Golden noise for diffusion models: A learning framework. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 17688–17697, 2025.

[12] C. Chen, L. Yang, X. Yang, L. Chen, G. He, C. Wang, and Y. Li. Find: Fine-tuning initial noise distribution with policy optimization for diffusion models. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 6735–6744, 2024.

[13] Y. Miao, W. Loh, P. Poupart, and S. Kothawade. A minimalist method for fine-tuning text-to-image diffusion models. *arXiv preprint arXiv:2506.12036*, 2025.

[14] D. Samuel, B. Meiri, H. Maron, Y. Tewel, N. Darshan, S. Avidan, G. Chechik, and R. Ben-Ari. Lightning-fast image inversion and editing for text-to-image diffusion models. *arXiv preprint arXiv:2312.12540*, 2023.

[15] Z. Qi, L. Bai, H. Xiong, and Z. Xie. Not all noises are created equally: Diffusion noise selection and optimization. *arXiv preprint arXiv:2407.14041*, 2024.

[16] D. Samuel, R. Ben-Ari, S. Raviv, N. Darshan, and G. Chechik. Generating images of rare concepts using pre-trained diffusion models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 4695–4703, 2024.

[17] R. Ron, G. Tevet, H. Sawdayee, and A. H. Bermano. Hoidini: Human-object interaction through diffusion noise optimization. *arXiv preprint arXiv:2506.15625*, 2025.

[18] Z. Tang, J. Peng, J. Tang, M. Hong, F. Wang, and T.-H. Chang. Inference-time alignment of diffusion models with direct noise optimization. *arXiv preprint arXiv:2405.18881*, 2024.

[19] A. Jordana, J. Zhang, J. Amigo, and L. Righetti. An introduction to zero-order optimization techniques for robotics. *arXiv preprint arXiv:2506.22087*, 2025.

[20] R. Y. Rubinstein. Optimization of computer simulation models with rare events. *European Journal of Operational Research*, 99(1):89–112, 1997.

[21] Z. Karnin, T. Koren, and O. Somekh. Almost optimal exploration in multi-armed bandits. In *International conference on machine learning*, pages 1238–1246. PMLR, 2013.

[22] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[23] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, S. Levine, J. Wu, C. Finn, H. Su, Q. Vuong, and T. Xiao. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.

[24] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine. Serl: A software suite for sample-efficient robotic reinforcement learning, 2024.

[25] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.

[26] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, et al. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[27] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021.

[28] Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. J. Fan, and Y. Zhu. Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16923–16930. IEEE, 2025.

[29] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine. Idql: Implicit q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.

[30] Z. Wang, J. J. Hunt, and M. Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2208.06193*, 2022.

[31] L. Yang, Z. Huang, F. Lei, Y. Zhong, Y. Yang, C. Fang, S. Wen, B. Zhou, and Z. Lin. Policy representation via diffusion probability model for reinforcement learning. *arXiv preprint arXiv:2305.13122*, 2023.

[32] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma. Learning a diffusion model policy from rewards via q-score matching. *arXiv preprint arXiv:2312.11752*, 2023.

[33] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.

[34] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021. URL https://arxiv.org/abs/2106.09685.

[35] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.

[36] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719*, 2025.

[37] Y. Luo, Z. Yang, F. Meng, Y. Li, J. Zhou, and Y. Zhang. An empirical study of catastrophic forgetting in large language models during continual fine-tuning, 2025. URL https://arxiv.org/abs/2308.08747.

[38] M. Tan and Q. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.

[39] Y. Wang, L. Wang, Y. Du, B. Sundaralingam, X. Yang, Y.-W. Chao, C. Pérez-D'Arpino, D. Fox, and J. Shah. Inference-time policy steering through human interactions. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 15626–15633. IEEE, 2025.

[40] M. Du and S. Song. Dynaguide: Steering diffusion polices with active dynamic guidance. *arXiv preprint arXiv:2506.13922*, 2025.

[41] M. Nakamoto, O. Mees, A. Kumar, and S. Levine. Steering your generalists: Improving robotic foundation models via value guidance. *arXiv preprint arXiv:2410.13816*, 2024.

[42] V. Heidrich-Meisner and C. Igel. Hoeffding and bernstein races for selecting policies in evolutionary direct policy search. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 401–408, 2009.

[43] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.

[44] H. Mania, A. Guy, and B. Recht. Simple random search provides a competitive approach to reinforcement learning. *arXiv preprint arXiv:1803.07055*, 2018.

[45] S. Bubeck, R. Munos, and G. Stoltz. Pure exploration in finitely-armed and continuous-armed bandits. *Theoretical Computer Science*, 412(19):1832–1852, 2011.

[46] D. Hendrycks. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

[47] D. P. Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[48] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.

[49] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 652–660, 2017.

[50] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[51] Z. Wang, Z. Li, A. Mandlekar, Z. Xu, J. Fan, Y. Narang, L. Fan, Y. Zhu, Y. Balaji, M. Zhou, et al. One-step diffusion policy: Fast visuomotor policies via diffusion distillation. *arXiv preprint arXiv:2410.21257*, 2024.

[52] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg. Consistency policy: Accelerated visuomotor policies via consistency distillation. *arXiv preprint arXiv:2405.07503*, 2024.

A Related Work

A.1 Lottery Ticket Hypothesis for Robot Control

We provide a detailed list of additional differences between the image generation setting, and the robot control setting.

1. **Dimensionality of denoising object**: In image generation, the object being denoised is an image, which typically contains 2 spatial dimensions and 3 channel dimensions for RGB color. For denoising an image, the dimensionality of the noise vector could be $128 \times 128 \times 3 = 49152$. In robotics, the object being denoised is an action chunk, whose dimensionality is equal to the product of the size of the one-step action and the chunk horizon length. An example of a particularly large action chunk is SmolVLA [26], which has a one-step action size of 32 and an action chunk horizon of 50, making the total dimensionality of the denoised object $50 \times 32 = 1600$. Therefore, the dimensionality of the denoising object in image generation can be orders of magnitude larger than the denoising object in robotics. For images, not all pixels are equally important, in the sense that some pixels will contain the foreground, target object, whereas the background pixels may be less important to capture for the target query. However, in robotics, there is no equivalent notion of "background" and "foreground" actions; instead, any action produced in the action chunk and executed by the robot may impact performance.

2. **Dimensionality of conditioning object**: In image generation, the object used for conditioning is typically a language description, which is either projected to a single vector or processed by a transformer into a sequence of discrete tokens. In robotics, the objects used for conditioning are typically visual data (e.g: images), proprioception data (e.g., joint positions, velocities, forces), and language (for task-conditioned policies like VLAs). Therefore, the dimensionality of the conditioning object can be orders of magnitude larger than the denoising object in image generation, and typically requires specialized encoders for the various conditioning objects. This makes designing observation-conditioned noise policies particularly challenging since they may need to handle a variety of robot sensor data.

3. **Temporal Decision Making**: Image generation tasks do not involve temporal decision making: once an image is generated, the next image to be generated is completely independent of the previous one. Additionally, for image generation, we may only need to optimize a noise for a single conditioning vector (i.e., just make good images of dogs). In robotics, we are addressing tasks that involve temporal decision making due to interacting in an environment, which means that the action chunk produced by the previous noise vector has large impact on distribution of next conditioning vectors we encounter. Unlike image generation where we may get similar prompts over time, in robotics, we may never get the same exact robot state to condition on again. Since the performance of the policy is not solely determined by any single action chunk, but instead depends on the performance throughout the entire task, evaluating tickets in robotics involves having a testing environment. Evaluating with an environment introduces unique complexities since much of the computational cost can come from the environment instead of running the policy, especially when the policy executes a large fraction of the action chunk before recomputing a new one.

4. **Cost of evaluating tickets**: In image generation, the metric to evaluate tickets is typically Fréchet Inception Distance (FID), which requires doing one full pass through the reverse process with the noise, making the model inference the most computationally expensive component. In robotics, we want to optimize the cumulative discounted expected rewards of the policy, which requires running the policy in an environment. For simulation, this causes evaluation to take more time and compute, but for hardware experiments, this is especially challenging and costly, potentially requiring human effort to deal with resets.

5. **Data manifold complexity**: For image generation tasks, the implicit data manifold that represents the space of desired images to be generated is high-dimensional, even when the text prompt is fixed. This is because there is typically a target concept to be generated in the foreground, which can be spatially placed in many locations, and the rest of the image is filled in with a background. This results in a combinatorially large number of ways to compose concepts in images. In robotics, the action chunk manifold is typically relatively low-dimensional (sometimes collapsing to just single actions depends on the conditioning vector and data). Therefore, many noises may map to the same action chunk for conditioning vectors, which makes it harder to get a differentiating signal on tickets. Some algorithms (like DSRL-NA [5]) exploit this aliasing property directly.

6. **Evaluation Metric Differentiability**: In image generation, differentiable metrics exists, such as FID score, which makes it possible to optimize the initial noise by calculating gradients through the model. In robotics, the rewards functions are typically not differentiable, therefore making it more challenging to implement gradient-based optimization techniques. However, methods like policy gradients [33], which avoid calculating gradients through the reward function, and transition dynamics can be leveraged to optimize cumulative discounted expected rewards.

A.2 Robot Policy Improvement Methods

One class of approaches to improving pretrained policies involves directly finetuning the model parameters (or applying model adaption methods like Low-Rank Adaptation (LoRA) [34]), using either supervised [35] or reinforcement learning (RL) losses [36]. These approaches apply the same pretraining techniques to improve policies, but can be challenging to use, since adjusting policy weights can lead to catastrophic forgetting [37] and is computationally hard for large models like VLAs. While some of these methods focus on RL training of auto-regressive models, more directly related to our work is DPPO [3], which backpropagates PPO gradients through the diffusion sampling process by treating an environment episode as an MDP that interleaves denoising and environment steps.

Another class of approaches learns a new model that adjusts policy output, such as residual policy learning [4] and latent steering [5]. These methods avoid adjusting pretrained policy weights, which helps preserve the original behavior, but require carefully designing/training new neural modules with sufficient model capacity to avoid acting as bottlenecks [38]. A last class of approaches involves making additional training/inference-time assumptions (e.g., inference-time steering [39], classifier-free guidance [7], world models [40], value-guidance [41]). These methods avoid adjusting the pretrained policy weights or learning additional neural modules for policy improvement, but they do not apply to all pretrained models (e.g., classifier-free guidance assumes a particular training regime) or assume extra models are available (e.g., access to a world model to use dynamics for guidance, or a user to provide corrective behaviors).

A.3 Direct Policy Search and Bandit-Based Evaluation

Our search methods belong to the long line of direct policy search, which treats policy performance as a black-box objective and optimizes it from rollout returns alone, without gradients through the policy or environment [42, 43, 44]. Policy gradient methods inject noise in action space and use the likelihood-ratio (REINFORCE [33]) estimator; this family injects noise in parameter space (here: latent-noise space) and uses finite-difference/smoothing estimators [43, 44] or rank-based selections [42, 20]. Evolution strategies scale this paradigm to deep network weights by trading sample efficiency for massive parallelism [43], while ARS shows that when the search space is small (static linear policies), even the simplest random search matches the sample efficiency of gradient-based deep RL [44]. Golden tickets continue this trajectory by searching over a latent input to a frozen generative policy, while showing the speed-up and parallelism of population-based search over reinforcement learning approaches (Section D.7).

A second connection is to pure-exploration bandits [45]: identifying the best ticket (or the elite set within a CEM iteration) from noisy Monte-Carlo returns under a fixed rollout budget is an instance of fixed-budget best-arm identification [21]. In our work, we show that our CEM search directly benefits from sequential halving [21] (Appendix B), eliminating poor tickets cheaply and concentrating evaluation on contenders. Methods involving confidence bounds [42] could further improve search efficiency and are left to future work.

B Search Methods

In this section, we describe three search strategies for finding golden tickets: random search (RS), the cross-entropy method (CEM), and zeroth-order search (ZOS). Algorithms 1, 2, and 3 show their pseudocode. All three optimize over the same initial-noise search space and use the same base policy, and rank candidates by mean evaluation return $\bar{R}_i$ across the search environments. We additionally describe a CEM variant that applies sequential halving to each iteration's population, evaluating candidates on a tiered schedule to reduce per-iteration env cost.

B.1 Random Search (RS)

Random Search [19] first samples N initial noise vectors (this is the main hyperparameter of our method), which we refer to as lottery tickets, and then evaluates each lottery ticket in a set of search environments M by performing policy rollouts. Each search environment is formulated as a single MDP; for single-task policies, only the starting state distribution may differ between search environments, whereas for multi-task policies, the reward function may also differ between environments. For each ticket, we run the policy in each environment, fixing the initial noise vector fed into the policy. We then calculate the average cumulative discounted rewards, providing an empirical estimate (i.e., Monte-Carlo estimate) of the value function for the ticket. After all tickets have been evaluated, we select and return the best performing ticket.

B.2 Cross-Entropy Method (CEM)

CEM [20] iteratively refines a Gaussian sampling distribution $\mathcal{N}(\mu, \sigma)$ over noise vectors, initialized at $\mu = 0$, $\sigma = 1$. At each of T iterations, we draw P candidate tickets $\epsilon_i = \mu + \sigma \odot z_i$ with $z_i \sim \mathcal{N}(0, I)$ and score each by its mean return $\bar{R}_i$ across the search environments M. The top-$K = \lfloor \rho P \rfloor$ elites by $\bar{R}_i$ are then used to refit μ and σ to their per-dimension mean and standard deviation, blended against the previous iterate by a smoothing factor α. To prevent premature distribution collapse, σ is floored at $\sigma_{\min}$ and optionally inflated by an extra term σ_{ex}. The best ticket seen across all iterations is returned as ϵ^*.

Sequential Halving. Vanilla CEM (Algorithm 2) scores every member of the P-candidate population on the same fixed pool of M search environments at every iteration, spending most of the env-evaluation budget on candidates that are obviously poor after only a handful of rollouts. We introduce a variant that keeps the CEM proposal-and-refit step unchanged but evaluates each iteration's population through a tiered schedule in the spirit of sequential halving for pure exploration in bandits [21]; we found this to substantially reduce the per-iteration env cost without harming elite identification.

The M env budget is partitioned into T_{tier} disjoint tiers of $M_0 = M/T_{\text{tier}}$ envs each (we use M_0=5 and T_{tier}=5, so M=25). Within a single CEM iteration:

- **Tier** 1**:** all P candidates roll out in tier-1's M_0 envs; rank by mean return $\bar{R}_i$.
- **Tier** $t \in \{2, \ldots, T_{\text{tier}}\}$**:** only the top half of the previous tier's survivors are rolled out in tier-t's M_0 fresh envs; each survivor's score is updated using *all* of its rollouts so far (cumulative $t \cdot M_0$ envs). Halving repeats until tier T_{tier}.

A candidate surviving every tier accumulates $T_{\text{tier}} \cdot M_0 = M$ env evaluations, while a candidate eliminated at tier t contributes only $t \cdot M_0$. The CEM update rule is otherwise unchanged: the top-

$K = \lfloor \rho P \rfloor$ elites are selected by each candidate's cumulative score, and μ, σ are refit as in vanilla CEM. All survivors at tier t roll out on the *same* M_0 env seeds and tier t's M_0 seeds are disjoint from those of tiers $1, \ldots, t-1$.

A flat-M CEM iteration costs $P \cdot M$ env rollouts. Under top-half halving, the iteration costs

$$P \cdot M_0 + \tfrac{P}{2} \cdot M_0 + \tfrac{P}{4} \cdot M_0 + \cdots + \tfrac{P}{2^{T_{\text{tier}}-1}} \cdot M_0 \;\approx\; 2 \cdot P \cdot M_0,$$

a roughly $T_{\text{tier}}/2$ sample-efficiency, while evaluating the deepest survivors at the same fidelity (M envs per survivor) as flat CEM.

B.3 Zeroth-Order Search (ZOS)

ZOS [19] performs a local random walk in noise space anchored at a pivot p, initialized as $p \sim \mathcal{N}(0, I)$. At each of T iterations, we construct K candidate tickets on the L_2-sphere of radius λ around p (directions $u/\|u\|_2$ with $u \sim \mathcal{N}(0, I)$), and additionally re-include the previously winning step $p+d$ when available. Each candidate is scored by its mean return $\bar{R}_\epsilon$ over M; the pivot advances to the best candidate $\epsilon^\dagger$ when $\bar{R}_{\epsilon^\dagger} > \bar{R}_p$ and we cache the corresponding step direction d for replay at the next iteration, otherwise the pivot stays put and d is cleared. As with CEM, the all-time best candidate is returned as ϵ^*.

Algorithm 1 Random Search (RS) for Optimizing Initial Noise

Input: pretrained policy π, environments M, reward function R, number of tickets N
Output: optimized initial noise vector ϵ^*
Sample $\{\epsilon_i\}_{i=1}^N \sim \mathcal{N}(0, I)$; initialize $\bar{R}_{\text{best}} \leftarrow -\infty$
for $i = 1$ to N **do**
    $S_i \leftarrow 0$
    for all $m \in M$ **do**
        Roll out π in m with fixed noise ϵ_i; obtain return R_m; $S_i \leftarrow S_i + R_m$
    $\bar{R}_i \leftarrow \frac{1}{|M|} S_i$
    if $\bar{R}_i > \bar{R}_{\text{best}}$ **then** $\bar{R}_{\text{best}} \leftarrow \bar{R}_i$; $\epsilon^* \leftarrow \epsilon_i$
return ϵ^*

Algorithm 2 Cross-Entropy Method (CEM) for Optimizing Initial Noise

Input: pretrained policy π, environments M, reward function R, population size P, iterations T, elite fraction ρ, smoothing α, std floor $\sigma_{\min}$, extra std σ_{ex}
Output: optimized initial noise vector ϵ^*
Initialize $\mu \leftarrow 0$, $\sigma \leftarrow 1$; $K \leftarrow \max(1, \lfloor \rho P \rfloor)$; $\bar{R}_{\text{best}} \leftarrow -\infty$
for $t = 1$ to T **do**
    Sample $\{z_i\}_{i=1}^P \sim \mathcal{N}(0, I)$; set $\epsilon_i \leftarrow \mu + \sigma \odot z_i$
    for $i = 1$ to P **do**
        $S_i \leftarrow 0$
        for all $m \in M$ **do**
            Roll out π in m with fixed noise ϵ_i; obtain return R_m; $S_i \leftarrow S_i + R_m$
        $\bar{R}_i \leftarrow \frac{1}{|M|} S_i$
        if $\bar{R}_i > \bar{R}_{\text{best}}$ **then** $\bar{R}_{\text{best}} \leftarrow \bar{R}_i$; $\epsilon^* \leftarrow \epsilon_i$
    $\mathcal{E} \leftarrow$ indices of the top-K candidates ranked by $\bar{R}_i$ (descending)
    $\mu \leftarrow \alpha \cdot \text{mean}\big(\{\epsilon_i : i \in \mathcal{E}\}\big) + (1 - \alpha)\, \mu$
    $\sigma \leftarrow \alpha \cdot \text{std}\big(\{\epsilon_i : i \in \mathcal{E}\}\big) + (1 - \alpha)\, \sigma$
    $\sigma \leftarrow \max(\sigma, \sigma_{\min})$
    if $\sigma_{\text{ex}} > 0$ **then** $\sigma \leftarrow \sqrt{\sigma^2 + \sigma_{\text{ex}}^2}$
return ϵ^*

Algorithm 3 Zeroth-Order Search (ZOS) for Optimizing Initial Noise

Input: pretrained policy π, environments M, reward function R, iterations T, candidates per iter K, radius λ
Output: optimized initial noise vector ϵ^*
Sample pivot $p \sim \mathcal{N}(0, I)$; evaluate $\bar{R}_p$; set $\epsilon^* \leftarrow p$, $\bar{R}_* \leftarrow \bar{R}_p$, $d \leftarrow \varnothing$
for $t = 1$ to T **do**
    Build $\mathcal{C}$: include $p + d$ if $d \neq \varnothing$, then fill to size K with $p + \lambda\, u/\|u\|_2$, $u \sim \mathcal{N}(0, I)$
    Evaluate each $\epsilon \in \mathcal{C}$; let $\epsilon^\dagger \leftarrow \arg\max_{\epsilon \in \mathcal{C}} \bar{R}_\epsilon$
    if $\bar{R}_{\epsilon^\dagger} > \bar{R}_p$ **then**
        $d \leftarrow \lambda\,(\epsilon^\dagger - p)/\|\epsilon^\dagger - p\|_2$
        $p \leftarrow \epsilon^\dagger$; $\bar{R}_p \leftarrow \bar{R}_{\epsilon^\dagger}$
    else
        $d \leftarrow \varnothing$
    if $\bar{R}_{\epsilon^\dagger} > \bar{R}_*$ **then** $\epsilon^* \leftarrow \epsilon^\dagger$; $\bar{R}_* \leftarrow \bar{R}_{\epsilon^\dagger}$
return ϵ^*

C Experimental benchmarks

Table 2 summarizes our benchmark suite. Per-benchmark policy and training details follow in the subsections below.

Table 2: Summary of experimental benchmarks. "ours" indicates a policy trained internally. Abbreviations: EE = end-effector, proprio = proprioception, PCD = pointcloud.

Benchmark	Policy / checkpoint	Obs / Action	Tasks
`franka_sim`	Flow-matching MLP (open-sourced)	state / EE-pos + gripper	`pick_cube`
robomimic	DPPO diffusion [3] `github.com/irom-princeton/dppo`	state / EE-pose	`Lift`, `Can`, `Square`, `Transport`
LIBERO	SmolVLA [26] `huggingface.co/HuggingFaceVLA/smolvla_libero`	2×RGB + state + lang / 32×50 chunk	30 tasks across OBJECT, GOAL, SPATIAL (10 each)
SimplerEnv (WidowX)	GR00T N1.5 [22] `github.com/NVIDIA/Isaac-GR00T/tree/n1.5-release`	RGB + lang / EE	`close_drawer`, `open_drawer`, `put_eggplant_in_basket`, `put_eggplant_in_sink`, `stack_cube`, `carrot_on_plate`, `spoon_on_towel`
DexMimicGen	U-Net diffusion (ours)	3×RGB + proprio / bimanual EE	`Drawer Cleanup`, `Tray Lift`, `Threading`, `Box Cleanup`, `Piece Assembly`
Franka HW (RGB)	U-Net diffusion (ours)	3×RGB + proprio / EE	`cube_pick`, `piston_assembly`
Franka HW (PCD)	U-Net diffusion (ours)	PCD + proprio / EE	`banana_pick`, `cup_push`

C.1 Flow matching policy in franka_sim

We use the franka_sim MuJoCo environment that was originally released in Luo et al. [24]. The task involves a single-arm Franka robot picking a cube; the cube spawns randomly within bounds of $[-0.25, 0.25]$ m (x) and $[0.25, 0.55]$ m (y) on the ground in front of the robot (roughly a $\frac{1}{2}$ m^2 region), and the goal is to lift it off the ground. We train a flow matching policy that takes as input the low-dimensional state of the cube and robot. The policy is a 4-layer MLP with GELU activations [46] and 256 hidden units per layer, and outputs an 8-step action chunk where each step contains a 3D end-effector position and a 1D gripper state (total chunk dimension $8 \times 4 = 32$). We collect

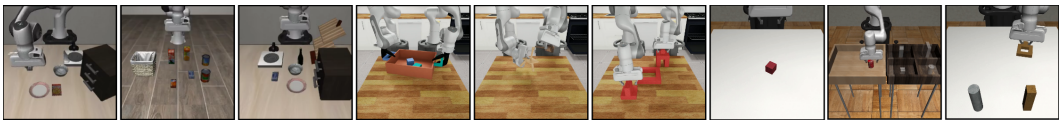


Figure 6: Sample images from some of our simulated benchmarks: (1-3) LIBERO-OBJECT, LIBERO-SPATIAL, and LIBERO-GOAL task suites, (4-6) DexMimicGen `Tray Lift`, `Threading`, and `Piece Assembly` tasks, and (7-9) robomimic `Lift`, `Can` and `Square` tasks.

1000 demonstrations using a task-and-motion planning heuristic, and train 4 model checkpoints for 100 epochs with a batch size of 20, using the Adam optimizer [47] with a learning rate of 0.001.

C.1.1 SmolVLA in LIBERO

We use the LIBERO benchmark [25], which contains multiple manipulation task suites that vary in object layouts, object types, and goals. We evaluate on 3 task suites of 10 tasks each: LIBERO-OBJECT, LIBERO-GOAL, and LIBERO-SPATIAL. The policy is SmolVLA [26], an open-source VLA from HuggingFace that conditions on images, robot state, and language; specifically we use the publicly released LIBERO-finetuned checkpoint (`https://huggingface.co/HuggingFaceVLA/smolvla_libero`, where all model and training details can be found) with all default inference configurations from the model card. The policy takes in 2 RGB images, the low-dimensional state of the robot, and a language description of the task, and outputs a one-step action of dimension 32 over a 50-step chunk horizon, for a total action chunk dimension of 1600.

C.1.2 DPPO in robomimic

We use the robomimic benchmark [27] and the same set of pretrained policies that were used in the original DSRL work [5]: specifically, diffusion policies trained using DPPO [3] on 4 separate tasks (`Lift`, `Can`, `Square`, `Transport`), which take as input low-dimensional state information about the objects and the robot. We use the publicly released checkpoints from the original DPPO codebase (also used in the original DSRL experiments): `https://github.com/irom-princeton/dppo`.

C.1.3 GR00T N1.5 in SimplerEnv (WidowX)

We evaluate the publicly released GR00T N1.5 checkpoint, a vision-language-action foundation model with a dual-system architecture in which a vision-language module interprets the RGB observation and the natural-language task description and a diffusion transformer generates the action chunk [22]. We evaluate it on the WidowX task suite of SimplerEnv [23], a simulator built on SAPIEN/ManiSkill2 that targets common WidowX manipulation behaviours; we use seven tasks where base performance was reported: `close_drawer`, `open_drawer`, `put_eggplant_in_basket`, `put_eggplant_in_sink`, `stack_cube`, `carrot_on_plate`, and `spoon_on_towel`. We run the provided GR00T N1.5 $\times$ SimplerEnv evaluation code (`https://github.com/NVIDIA/Isaac-GR00T/tree/n1.5-release/examples/SimplerEnv`), with the default inference configuration shipped in the example for a fair comparison.

C.1.4 RGB diffusion policy in DexMimicGen

We use the DexMimicGen benchmark [28], which involves five bimanual tasks with Franka arms: `Drawer Cleanup`, `Tray Lift`, `Threading`, `Box Cleanup`, and `Piece Assembly`. For each of the 5 tasks we train a separate diffusion policy with a U-Net backbone [48], a ResNet-18 encoder for the RGB images, and an MLP for the robot proprioception data. The policy takes in 3 RGB images and the end-effector position, quaternion, and gripper state for both arms.

C.1.5 Franka hardware - RGB diffusion policy

We use Franka hardware to conduct real-world experiments: the Franka Research 3 arm is equipped with a Robotiq 2F-85 gripper, with RealSense D435 cameras for the two static external cameras and a D405 for the wrist camera. We train RGB diffusion policies (one wrist camera, two static external cameras, and proprioception data) via behavior cloning on a block picking task(Figure 2a) and piston-assembly task. Both policies uses a standard diffusion policy architecture with a U-Net backbone and ResNet-18 image encoders. We optimize a binary success signal as the reward function: for block picking, it is whether the cube is lifted off the table, and for piston assembly, it is based on whether the piston head and rod are properly mated for subsequent pin insertion.

C.1.6 Franka hardware - Pointcloud diffusion policy

We use the same Franka hardware described above, but with two pointcloud policies for two separate tasks: (1) picking a banana (Figure 2c) and (2) pushing a cup to the center of the table (Figure 2b). For the pointcloud policies, we use the same U-Net backbone as in the RGB setup but replace the image encoder with a PointNet encoder [49] for the pointcloud. We use 2 calibrated extrinsic cameras to generate a single fused pointcloud, and remove all points that are at or below the surface of the table. We again optimize a binary success signal as the reward function.

D Results

This appendix's results section roughly follows the order of the main paper: existence of golden tickets (Section D.1), supporting statistics (Sections D.2 and D.3), real-robot results (Section D.4), effect of DDIM steps (Section D.5), search-method comparison (Section D.6), detailed comparison with DSRL (Section D.7), and multi-task policy improvement (Section D.8).

D.1 Per-task golden ticket vs. Gaussian noise comparison

This lets us understand whether golden tickets exist at all, and what their best possible performance is regardless of computational or sampling cost. If the best lottery tickets are often worse than sampling from a Gaussian on held-out test environments, then the lottery ticket hypothesis cannot reliably be used to improve robot control policies. Due to the continuous and high-dimensional nature of the noise vectors, it is infeasible to test all possible tickets, but we attempt to search for as many tickets as possible to get an approximate upper bound. Search budgets vary across benchmarks; per-benchmark details are reported alongside the corresponding results below.

Golden tickets outperform Gaussian noise in 46 out of 51 tasks, and at least match it in 49: Table 3 summarizes per-benchmark base and best-golden-ticket performance, per-task search budget, and search method. Overall, our results demonstrate that the lottery ticket hypothesis is often true, in the sense that golden tickets can likely be found that improve over the base policy performance.

Table 3: Per-task summary of the golden-ticket vs. Gaussian comparison across all 51 task–policy pairs. **Base** and **Golden** are held-out success rates (%); All **Golden** tickets are found using random search, except for SimplerEnv which uses CEM. **Search** combines the per-task budget (tickets × search-envs per ticket) with the search method. Shaded *Avg.* rows give the per-benchmark mean across the listed tasks. **Note**: better performing golden tickets can be found for robomimic using CEM, but have not been included in the existence results.

Benchmark	Task	Base	Golden	Search
`franka_sim`	*Avg. over* 4 *ckpts*	38.5 ± 27.9	96.0 ± 2.4	$\approx 250 \times 25$ (RS)
robomimic	`Lift`	78.4 ± 3.9	96.2 ± 1.4	
	`Can`	42.8 ± 4.4	80.8 ± 4.6	
	`Square`	52.2 ± 3.6	32.9 ± 6.7	5000×100 (RS)

continued on next page

Table 3 continued from previous page.

Benchmark	Task	Base	Golden	Search
	`Transport`	10.7 ± 2.2	18.8 ± 3.6	
	Avg.	46.0	57.2	
LIBERO-Object	T0	82	100	1416×5 (RS)
	T1	98	100	
	T2	98	100	
	T3	98	100	
	T4	76	100	
	T5	82	100	
	T6	100	100	
	T7	90	100	
	T8	96	100	
	T9	100	100	
	Avg.	92.0	100.0	
LIBERO-Goal	T0	72	100	1338×3–5 (RS)
	T1	94	100	
	T2	86	100	
	T3	52	68	
	T4	92	100	
	T5	78	100	
	T6	80	94	
	T7	100	100	
	T8	94	100	
	T9	68	90	
	Avg.	81.6	95.2	
LIBERO-Spatial	T0	78	84	1081×3–5 (RS)
	T1	94	98	
	T2	84	92	
	T3	62	100	
	T4	84	92	
	T5	58	90	
	T6	90	100	
	T7	90	96	
	T8	78	90	
	T9	86	92	
	Avg.	80.4	93.4	
SimplerEnv (WidowX)	`close_drawer`	65	100.0	$250 \times (5$–$10)$ (CEM)
	`open_drawer`	84	97.3	
	`put_eggplant_in_basket`	63	96.0	
	`put_eggplant_in_sink`	21	76.3	
	`stack_cube`	54	73.7	
	`carrot_on_plate`	72	93.0	
	`spoon_on_towel`	82	90.0	
	Avg.	63.0	89.5	—
DexMimicGen	`Drawer Cleanup`	82.2 ± 4.4	88.0 ± 2.9	500×50 (RS)
	`Tray Lift`	71.0 ± 3.9	79.9 ± 4.6	
	`Threading`	62.0 ± 4.1	60.0 ± 4.9	
	`Box Cleanup`	87.6 ± 2.4	97.8 ± 1.1	
	`Piece Assembly`	68.3 ± 3.3	75.6 ± 3.1	
	Avg.	74.2	80.3	
Franka HW	`cube_pick`	80	98	6×25 (RS)

continued on next page

Table 3 continued from previous page.

Benchmark	Task	Base	Golden	Search
	piston_assembly	60	88	22 tickets, 61 eps (RS)
	banana_pick	50	68	10×5 (RS)
	cup_push	40	100	10×5 (RS)
	Avg.	57.5	88.5	—

franka_sim. For each of the 4 model checkpoints, we search for $\approx$ 250 lottery tickets across 25 starting block poses and evaluate on 25 held-out evaluation block poses. franka_sim is the most extreme example, where the base policy performance for cube picking have an average success rate of 38.5%, whereas the best golden tickets have an average success rate of 96%.

robomimic. We search for 5000 tickets per task with 100 environments each, and evaluate on 100 episodes across 5 random seeds. Golden tickets were found for 3 out of 4 of the tasks (`Lift`, `Can`, and `Transport`), whereas there is only one task (`Square`) for which we did not find a ticket that outperformed the base policy. The most extreme performance improvements are in `Can`, where average base policy success rate is 42.8%, and is improved to 80.8% with golden tickets, resulting in a 38% improvement.

LIBERO. We searched for 1416 tickets in LIBERO-OBJECT, 1338 tickets in LIBERO-GOAL, and 1081 tickets in LIBERO-SPATIAL; for LIBERO-OBJECT we used 5 search environments per ticket, whereas for the other two task suites we varied between 3 and 5 search environments. For each task suite, we report the average performance of the best performing tickets for each of the tasks in the task suite, emulating a setting where we optimize tickets in a per-task setting (See Table 1 for details). We evaluate golden tickets and the base policy on 50 episodes per task. [4] In this setting, we are able to find golden tickets that boost the performance of the base policy in all three task suites: 13% increase for LIBERO-SPATIAL, 12.8% for LIBERO-GOAL, and 8% for LIBERO-OBJECT.

SimplerEnv (WidowX). We search for golden tickets with CEM (population P=20, elite fraction ρ=0.3, extra std σ_{ex}=0.1, episode length 120 steps, hard cap of 250 candidates per seed, early-stopping after 3 perfect-score tickets). Each candidate is scored on 5 search-environment rollouts for `close_drawer`, `put_eggplant_in_basket`, `put_eggplant_in_sink`, and `stack_cube`, and on 10 rollouts for `carrot_on_plate`, `spoon_on_towel`, and `open_drawer`. For each task we run the search across 3 seeds, evaluate the top-3 tickets per seed on 300 held-out episodes; base-policy numbers are taken from the public NVIDIA GR00T N1.5 SimplerEnv evaluation [22]. The **existence** of golden tickets can be shown for all 7 tasks, with the largest gain on `put_eggplant_in_sink` (21%→76.3%, +55.3%).

DexMimicGen. We search for 500 tickets across 50 environments each, and evaluate on 100 environments across 5 random seeds. Golden tickets were found in 4 out of 5 tasks (`Drawer Cleanup`, `Tray Lift`, `Box Cleanup`, and `Piece Assembly`). Overall, the performance gains of the best golden ticket are milder, with the biggest gap coming from `Box Cleanup` where the average success rate of the base policy and best golden tickets perform at 87.6% and 97.8% respectively. The one task for which we did not find a golden ticket, `Threading`, has relatively similar average success rate between the base policy (62%) and the best lottery ticket (60%).

Hardware experiments. Complete results for our block picking RGB policy can be found in Table 5. When using standard Gaussian noise, our block-picking policy successfully picks up the cube 80% of the time (40 out of 50). After searching for 6 tickets, each with 25 episode rollouts, 150 episodes in total, we find a golden ticket (Ticket 5) that succeeded 25 out of 25 times, and also a ticket (Ticket 6) that succeeded only 1 out of 25 times). When we then evaluate those two tickets an additional 50 times, Ticket 5 successfully picked the cube 98% of the time (49 out of 50), where Ticket 6 only succeeded 2 out of 50 times. We therefore show we can drive success rate from 80%

[4] Our reported numbers on SmolVLA's success rate for the publicly released checkpoint differs by $\approx$ 5% from the original paper's [26] reported results. This is because Shukor et al. [26] evaluates on 10 episodes for each LIBERO task suite, whereas we do 50 since 10 was not sufficiently reliable.

to 98%, an 18% increase, using just 150 search episodes, or conversely steer the policy to miss with extreme reliability.

For the piston-assembly policy, we evaluated the base policy 25 episodes, where it completed the assembly 60% of the time. After searching across 22 tickets (totaling 61 episodes), we found a ticket that performed 88% when evaluated on 25 episodes. This results in a 28% success rate improvement for the longest-horizon and most dexterous real-world task.

For the banana picking pointcloud policy, we initially evaluate the base policy at a single location 10 times, where it successfully picks the banana 30% of the time. After searching for 10 tickets in the same location for 5 episodes each, we found a ticket that performed 100%. We then evaluated that golden ticket and the base policy at 4 other locations on the table, 10 episodes each. The base policy has an average success rate of 50% whereas the golden ticket performs at 68%. This results in a 18% improvement across the table after only 50 episodes.

For the cup pushing pointcloud policy, we evaluated the base policy 10 times and observed an average success rate of 40%. We searched for 10 tickets, with 5 rollouts, and found a ticket that achieved 100% success rate. When evaluated an additional 10 times, we found that it continued to achieve a 100% success rate. This constitutes our largest improvements on hardware, where the average success rate was increased by 60%.

D.2 Statistical significance of the improvement rate

For each of the $n = 51$ tasks i, we define a binary outcome $X_i = 1$ if the best searched noise vector strictly outperforms the base (Gaussian noise) policy on the held-out evaluation, and $X_i = 0$ otherwise. Modeling $X_i \overset{\text{iid}}{\sim} \text{Bernoulli}(p)$, we test

$$H_0 \colon\ p \le 0.5 \quad \text{vs.} \quad H_1 \colon\ p > 0.5,$$

where p is the probability that, for a task drawn from the distribution sampled by our benchmark suite (franka_sim $\cup$ LIBERO $\cup$ robomimic $\cup$ DexMimicGen $\cup$ SimplerEnv (WidowX) $\cup$ real-world), the best searched noise vector strictly improves over Gaussian sampling. The null $p_0 = \frac{1}{2}$ corresponds to the standard sign-test null.

Letting $K = \sum_i X_i$, the exact one-sided p-value is $\mathbb{P}_{p=0.5}(K \ge k_{\text{obs}}) = 2^{-n} \sum_{j=k_{\text{obs}}}^{n} \binom{n}{j}$. For $k = 46$ we obtain p-value $\approx 1.16 \times 10^{-9}$, and for the weaker "at-least-match" count $k = 49$, p-value $\approx 5.89 \times 10^{-13}$. We report a Wilson 95% score interval for p,

$$\text{CI} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + z^2/n}, \qquad z = 1.96,$$

which gives $[0.79, 0.96]$ for $\hat{p} = 46/51$ and $[0.87, 0.99]$ for $\hat{p} = 49/51$.

Approximations. (i) Tasks within a benchmark may share a base policy and also the observation pipeline (e.g., all 30 LIBERO tasks use SmolVLA), so the iid assumption is approximate (ii) The inference applies to the population sampled by our four benchmarks, not to all manipulation tasks. (iii) The indicator X_i does not weight by the per-task uncertainty in the success-rate estimate

D.3 Performance improvement with golden tickets

Beyond whether golden tickets improve over Gaussian sampling (Appendix D.2), we ask: *above what base policy performance do golden tickets reliably improve task success?* Concretely, we seek the lowest base policy floor $T^\star$ such that, among the task–policy pairs whose base success rate is at least $T^\star$, at least 90% are pushed past a high absolute success bar by their golden ticket. We instantiate "high" at two levels- $G \in \{90, 97\}\%$.

For a base floor T and absolute bar G, let

$$\hat{p}(T, G) \ = \ \frac{\#\{\text{pairs with base} \ge T \text{ and gold} \ge G\}}{\#\{\text{pairs with base} \ge T\}}$$

be the empirical pass rate among the pairs included by floor T. We then report

$$T_G^\star \;=\; \min\{\, T : \hat{p}(T', G) \geq 0.90 \;\text{ for every } T' \geq T \,\},$$

i.e., the lowest base floor above which the empirical pass rate *stays* at $\geq 90\%$.

Table 4: Base policy floor $T_G^\star$ above which golden tickets reach the absolute bar G in at least 90% of the supported task–policy pairs. n is the number of pairs with base $\geq T_G^\star$; k is the number of those reaching $\geq G$.

Target G	$T_G^\star$	Support n	Successes k	k/n
90%	71%	34	31	0.912
97%	87.6%	15	14	0.933

Base floor $T^\star = 71\%$ (bar $G = 90\%$, $n = 34$). The 34 pairs with base $\geq 71\%$ are (with 3 trivial passes where base policy performance is 100%):

- LIBERO-SPATIAL (8): `T0`, `T1`, `T2`, `T4`, `T6`, `T7`, `T8`, `T9` (excludes `T3`, `T5`)
- LIBERO-GOAL (8): `T0`, `T1`, `T2`, `T4`, `T5`, `T6`, `T7`, `T8` (excludes `T3`, `T9`)
- LIBERO-OBJECT (10): all tasks `T0`–`T9`
- DexMimicGen (3): `Drawer Cleanup`, `Tray Lift`, `Box Cleanup`
- robomimic (1): `Lift`
- Real robot (1): cube picking
- SimplerEnv (WidowX) (3): `carrot_on_plate`, `spoon_on_towel`, `open_drawer`

Three of these 34 pairs do not reach $\geq 90\%$:

- DexMimicGen `Tray Lift`: base 71%, gold 79.9%
- DexMimicGen `Drawer Cleanup`: base 82.2%, gold 88%
- LIBERO-SPATIAL `T0`: base 78%, gold 84%

Base floor $T^\star = 87.6\%$ (bar $G = 97\%$, $n = 15$). The 15 pairs with base $\geq 87.6\%$ are:

- LIBERO-SPATIAL (3): `T1`, `T6`, `T7`
- LIBERO-GOAL (4): `T1`, `T4`, `T7`, `T8`
- LIBERO-OBJECT (7): `T1`, `T2`, `T3`, `T6`, `T7`, `T8`, `T9`
- DexMimicGen (1): `Box Cleanup`

One of these 15 pairs does not reach $\geq 97\%$:

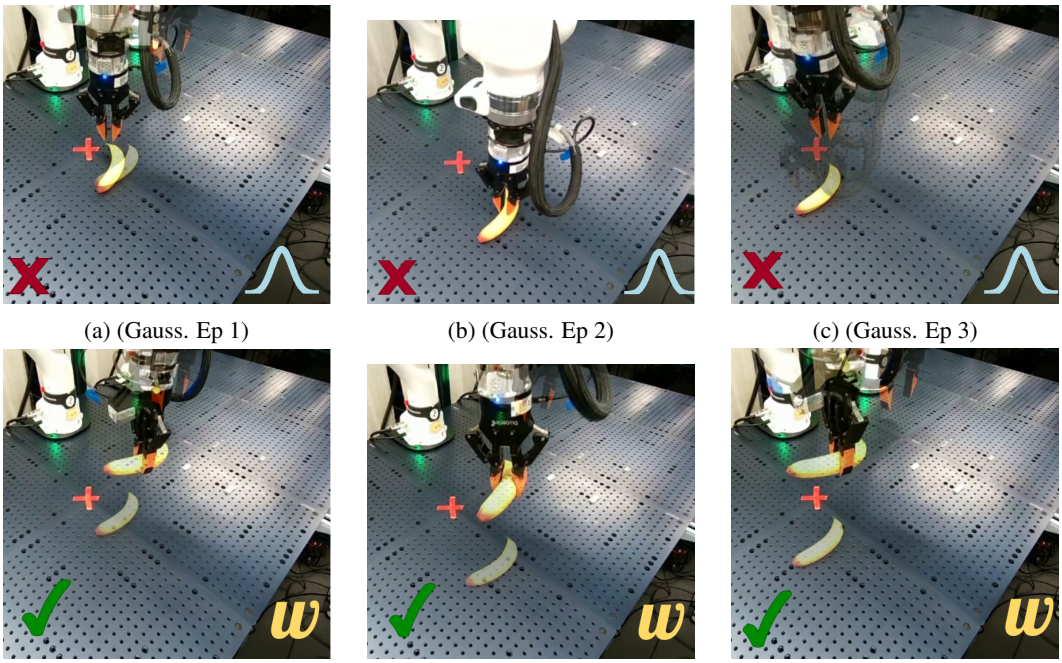
- LIBERO-SPATIAL `T7`: base 90%, gold 96%

D.4 Real World and SimplerEnv Results

We present complete experimental results (the number of tickets searched and evaluated) for the cube picking, banana picking, and cup pushing tasks in Tables 5, 7, and 6 respectively.

For cup pushing, the base policy was evaluated over 10 episodes. We searched 10 tickets, each for 5 episodes, and then the best performing ticket (Ticket 2) and evaluated an additional 10 episodes.

For banana picking, the base policy was evaluated in 5 locations, 10 episodes each. We searched 10 tickets, each for 5 episodes, in one location, then took the best performing ticket and evaluated it in 4 other locations for 10 episodes each. We note that Ticket #1 performs worse than the base policy in two locations (#2 and #3), these two locations are significantly further to the right side of the table than locations #1, #4, and #5. While this suggests that golden tickets can generalize within



(a) (Gauss. Ep 1) (b) (Gauss. Ep 2) (c) (Gauss. Ep 3)

(d) (G.T. Ep 1) (e) (G.T. Ep 2) (f) (G.T. Ep 3)

Figure 7: (a-c) A diffusion policy trained to pick a banana across the table, with three different failure spots shown. Every time it produces an action chunk, random initial noise is sampled from an isotropic Gaussian. (d-f) **With the same network and weights**, we can adapt this policy to successfully pick the banana from all locations, by instead using a constant initial noise vector called a *golden ticket (G.T.)*. We optimize initial noise vectors to steer pretrained policies to maximize downstream rewards.

Table 5: Success rates for base policy and lottery tickets for hardware cube picking task. The base policy was evaluated over 50 episodes. We searched 6 tickets, each for 25 episodes, and then took the two most extreme results (Tickets 5 and 6) and additionally evaluated over 50 episodes.

Policy	**Search Success Rate**	**Eval Success Rate**
Base Policy	–	40/50 (80%)
Ticket 1	11/25 (44%)	–
Ticket 2	24/25 (96%)	–
Ticket 3	10/25 (40%)	–
Ticket 4	10/25 (40%)	–
Ticket 5	25/25 (100%)	49/50 (98%)
Ticket 6	1/25 (4%)	2/48 (4%)

some convex hull of spatial locations, complete coverage of the table may require the use of multiple tickets across the space.

For SimplerEnv, we search for around 250 tickets for each task individually for 3 different seeds, with 5 evaluations per ticket for `close_drawer`, `put_eggplant_in_basket`, `put_eggplant_in_sink`, and `stack_cube`, and 10 evaluations per ticket for `carrot_on_plate`, `spoon_on_towel`, and `open_drawer`. For each task, we stop searching when we land 3 tickets with perfect scores in the search environments. After the search, we evaluate the top 3 tickets per seed on 300 held-out episodes. Base policy numbers are taken from the public NVIDIA GR00T N1.5 SimplerEnv evaluation [22]. The complete results for each seed and task can be found in Table 8 for the best of the top-3 tickets and in Table 9 for the best ticket based on the search rewards.

Table 6: Success rates for base policy and lottery tickets for hardware cup pushing task.

Policy	Search Success Rate	Eval Success Rate
Base Policy	–	4/10 (40%)
Ticket 1	1/5 (20%)	–
Ticket 2	5/5 (100%)	10/10 (100%)
Ticket 3	0/5 (0%)	–
Ticket 4	5/5 (100%)	–
Ticket 5	0/5 (0%)	–
Ticket 6	0/5 (0%)	–
Ticket 7	1/5 (25%)	–
Ticket 8	5/5 (100%)	–
Ticket 9	5/5 (100%)	–
Ticket 10	5/5 (100%)	–

Table 7: Success rates for base policy and lottery tickets for banana picking task.

Policy	Location	Search Success Rate	Eval Success Rate
Base Policy	#1	–	3/10 (30%)
Base Policy	#2	–	8/10 (80%)
Base Policy	#3	–	5/10 (50%)
Base Policy	#4	–	1/10 (10%)
Base Policy	#5	–	8/10 (80%)
Ticket 1	#1	5/5 (100%)	5/5 (100%)
Ticket 1	#2	–	4/10 (40%)
Ticket 1	#3	–	0/10 (0%)
Ticket 1	#4	–	10/10 (100%)
Ticket 1	#5		8/10 (80%)
Ticket 2	#1	1/5 (20%)	–
Ticket 3	#1	1/5 (20%)	–
Ticket 4	#1	0/5 (0%)	–
Ticket 5	#1	0/5 (0%)	–
Ticket 6	#1	0/5 (0%)	–
Ticket 7	#1	0/5 (0%)	–
Ticket 8	#1	0/5 (0%)	–
Ticket 9	#1	0/5 (0%)	–
Ticket 10	#1	0/5 (0%)	–

Interestingly, we find that for some tasks such as `close_drawer` and `put_eggplant_in_basket`, we are able to find golden tickets in fewer than 50 tickets ($50 \times 5 = 250$ episodes) across all 3 seed trials, implying that CEM is easily able to improve the policy for these tasks by around 30% to near perfection (99.2% and 91.7% each) with less than 250 policy evaluations.

CEM is also able to find tickets that improve held-out performance within roughly the 250-ticket budget for `put_eggplant_in_sink`, `stack_cube`, and `open_drawer`, raising their averaged Gold success rates by $+48.3\%$, $+17.3\%$, and $+11.7\%$ respectively over the base policy (Table 8). For `carrot_on_plate` and `spoon_on_towel`, however, CEM finds tickets that improve held-out performance on some seeds but underperform the base policy on others, despite the search terminating early after landing 3 tickets with perfect scores in the search environments. This indicates that the per-ticket Monte-Carlo estimate of policy performance is noisy on these tasks, and a larger per-ticket evaluation budget would be needed to reliably select tickets that generalize to the held-out distribution. The first perfect ticket (the first ticket returned by each seed's search, ranked by search-environment reward) improves the held-out success rate over the base policy by $+11.6\%$ on average. Choosing the ticket with the highest search reward (after collecting 3 perfect tickets;

reward accounting for episode length) improves the held-out success rate over the base by $+14.8\%$ on average, with detailed results in Table 9.

Table 8: SimplerEnv (WidowX) per-task results for search seeds. For each (task, seed), **Gold (Best)** is the best held-out success rate ($\%$) among the top-3 tickets returned by that seed's CEM run; N is the number of CEM candidates evaluated in that run before early-stopping. **Avg. Gold (Best)** is the mean of the three per-seed Gold (Best) values.

		Avg.	Seed 999		Seed 1000		Seed 1001	
Task	**Base**	**Gold (Best)**	**Gold (Best)**	N	**Gold (Best)**	N	**Gold (Best)**	N
`close_drawer`	65	99.2	99.0	11	98.7	16	100.0	6
`put_eggplant_in_basket`	63	91.7	96.0	16	93.3	32	85.7	14
`put_eggplant_in_sink`	21	69.3	67.7	270	64.0	27	76.3	100
`stack_cube`	54	71.3	71.7	230	73.7	69	68.7	27
`carrot_on_plate`	72	74.3	78.7	150	51.3	106	93.0	81
`spoon_on_towel`	82	79.9	77.3	230	72.3	53	90.0	66
`open_drawer`	84	95.7	96.0	183	97.3	121	93.7	144

Table 9: SimplerEnv WidowX per-task results for the highest-reward ticket returned by each seed's CEM search. **Gold** is the held-out success rate ($\%$) on 300 episodes from each seed's CEM run. N is the number of CEM candidates evaluated before early-stopping. **Avg. Gold** is the mean of the three per-seed Gold values.

		Avg.	Seed 999		Seed 1000		Seed 1001	
Task	**Base**	**Gold**	**Gold**	N	**Gold**	N	**Gold**	N
`close_drawer`	65	95.6	97.7	11	89.0	16	100.0	6
`put_eggplant_in_basket`	63	89.8	96.0	16	92.7	32	80.7	14
`put_eggplant_in_sink`	21	56.0	67.7	270	24.7	27	75.7	100
`stack_cube`	54	66.8	66.7	230	65.0	69	68.7	27
`carrot_on_plate`	72	70.8	72.3	150	47.0	106	93.0	81
`spoon_on_towel`	82	75.7	75.3	230	66.0	53	85.7	66
`open_drawer`	84	90.3	87.3	183	93.3	121	90.3	144

D.5 Effect of DDIM Steps on Lottery Ticket Hypothesis

When using diffusion models, our proposed lottery ticket search, along with latent steering approaches like DSRL [5] assume DDIM sampling [8] since it is deterministic and therefore takes less samples to estimate the cumulative discounted expected rewards induced by an initial noise vector. DDIM sampling has been widely adopted in robotics as an alternative to DDPM [50] as it requires fewer sampling steps, although other techniques such as distillation [51, 52] have additionally been proposed. Since our work only uses DDIM, we specifically investigate how changing the number of denoising steps in DDIM impacts performance of the base policy and golden tickets. However, we suspect these results have implications on the other aforementioned sampling techniques.

Figure 8 and 9 for robomimic and DexMimicGen respectively compare the average performance of top-3 golden tickets with the base policy for 2 and 8 DDIM sampling steps. We see that golden tickets found for DDIM-2 match or outperform the base policy (using DDIM-2) for all tasks across both benchmarks. Note that the golden tickets for 2 and 8 DDIM steps are searched separately and with a budget of 5000 and 500 tickets for robomimic and DexMimicGen, respectively. We find that DDIM-2 golden tickets close the performance gap with the base policy using DDIM-8 in several tasks such as `Lift`, `Can` and `Box Cleanup`.

Interestingly, across Figures 8 and 9, we see that golden tickets yield bigger performance improvements as compared to the base policy at 2 DDIM steps than at 8 DDIM steps. Mechanistic explanations for golden tickets are complicated by the temporal nature of policy rollouts, which we leave for future work to investigate further.

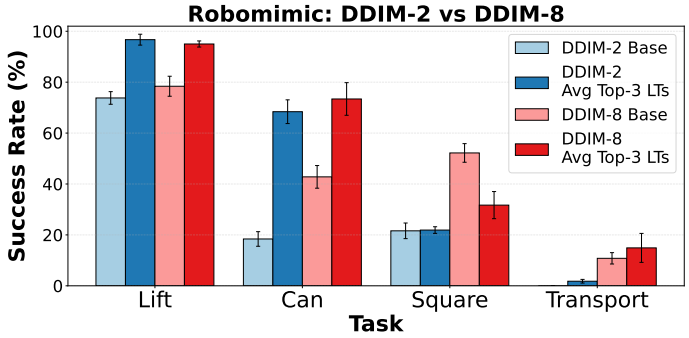


Figure 8: Base policy and golden ticket performance with 2 and 8 DDIM steps for 4 tasks in robomimic. Golden tickets found for `Lift` and `Can` at DDIM-2 even outperform the base policy performance with 8 DDIM steps.

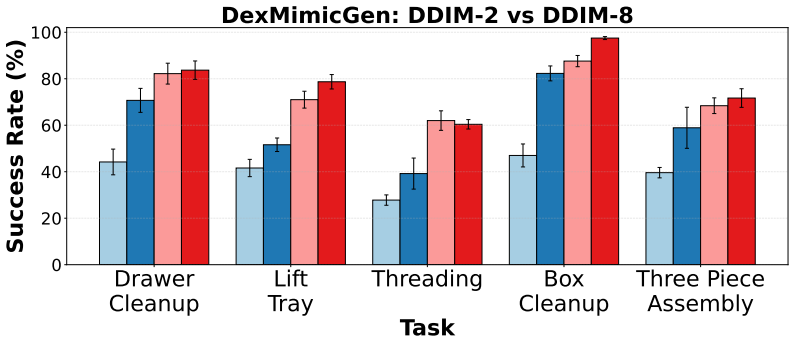


Figure 9: Base policy and golden ticket performance with 2 and 8 DDIM steps for 5 tasks in DexMimicGen. We see a greater performance increase from golden tickets at DDIM-2 as compared to DDIM-8. The golden tickets for both DDIM-steps were found with the equal search budgets.

D.6 Search Analysis

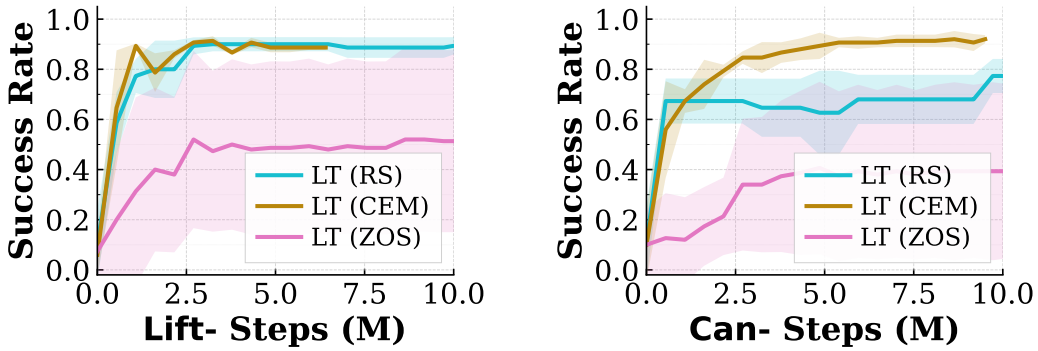
D.6.1 Search Method Comparison

We compare RS (Algorithm 1), CEM (Algorithm 2), and ZOS (Algorithm 3) on robomimic `Lift` and `Can`, all sharing the same initial-noise search space and base policy. Hyperparameters used to produce the curves in Figure 10: *RS* samples noise from $\mathcal{N}(\mathbf{0}, I)$ with 20 search environments per ticket; *CEM* uses population $P=50$, iterations $T=50$, elite fraction $\rho=0.2$, initial $\sigma=1$, smoothing $\alpha=1$, std floor $\sigma_{\min}=0.01$, and 20 search environments per candidate; *ZOS* uses $T=250$ iterations with $K=10$ candidates per iteration on an L_2-sphere of radius $\lambda=0.5$, and 20 search environments per candidate. Each method is run for 3 random seeds; Figure 10 reports mean $\pm$ standard deviation of eval success rate.

D.6.2 Top-50 Tickets and Stochastic Sampling

Figure 11 compares the per-ticket eval success rate of the top-50 tickets returned by RS and CEM on robomimic `Can`. Although RS was given a higher search-time budget per ticket for this experiment(50 episodes vs. CEM's 20, for a tighter Monte-Carlo estimate of the per-ticket return), CEM still finds many more high-performing tickets; RS quality decays sharply with rank because RS samples from the prior at every iteration while CEM adapts its sampling distribution to the elites found so far.

Sampling the policy's initial noise uniformly from the top-k CEM tickets at every action step also sustains performance up to $k=50$, indicating that golden tickets are not isolated anomalies but lie in a broader high-reward region of the noise space. This also has implications in terms of stochastic



(a) robomimic `Lift` (b) robomimic `Can`

Figure 10: Comparison of the three search methods – random search (RS), cross-entropy method (CEM), and zeroth-order search (ZOS) – on robomimic `Lift` (a) and `Can` (b). Curves report mean $\pm$ standard deviation across 3 seeds.

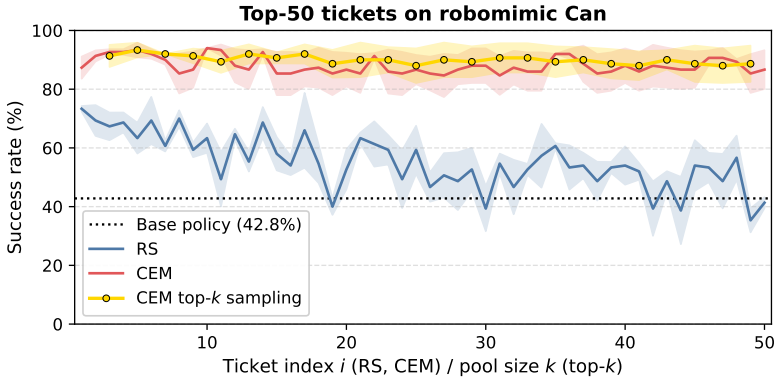


Figure 11: Per-ticket success rate of the top-50 tickets returned by RS and CEM on robomimic `Can` as returned by search. The dotted black line marks the base policy success rate (42.8%). The orange curve shows the performance of top-k CEM tickets randomly sampled for each action generation step, with the x axis denoting k. CEM and CEM top-k sustain performance till 50 tickets while RS deteriorates as the order of tickets increase.

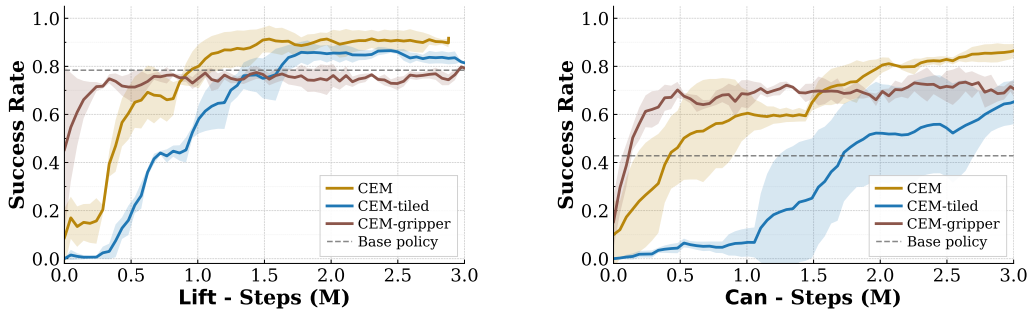
sampling using golden tickets, as we show that it does not result in loss of performance, while leaving an analysis of the resulting mulitmodal coverage to future work.

D.6.3 Search-Subspace Ablation: Tiling and Gripper-Only Noise

We next study how restricting CEM's proposal distribution to lower-dimensional subspaces of the noise vector affects search efficiency and final performance. On ROBOMIMIC the per-step action is 7-dimensional (3 end-effector translation +3 rotation +1 gripper) and the policy returns a 4-step action chunk, so the full noise vector has dimension 28. We compare three variants of CEM, all sharing every other hyperparameter and the same per-ticket evaluation setting:

- **CEM** (*full*, 28-dim): CEM proposes the entire noise vector freely.
- **CEM-tiled** (7-dim): CEM proposes a single 7-dim per-step noise vector that is then tiled across all 4 chunk steps, enforcing temporal sharing within an action chunk.
- **CEM-gripper** (4-dim): CEM proposes only the 4 gripper-channel entries (one per chunk step); the remaining 24 body dimensions are sampled fresh from $\mathcal{N}(\mathbf{0}, I)$ at each rollout.

Figure 12 reveals how tiling and gripper-only "sub-ticket" search affect the policy performance. **CEM-gripper** converges quickly on both tasks, but plateaus below full-CEM: the gripper channel



(a) `Lift` (b) `Can`

Figure 12: Held-out success rate versus environment steps for CEM operating in three different noise subspaces on ROBOMIMIC `Lift` (left) and `Can` (right): full 28-dim noise (**CEM**, gold), 7-dim per-step noise tiled across the 4-step chunk (**CEM-tiled**, blue), and 4-dim gripper-only noise with body dimensions sampled from $\mathcal{N}(\mathbf{0}, I)$ (**CEM-gripper**, brown). The dashed line marks base-policy success rate. Shaded regions show ± 1 standard deviation across 3 seeds.

alone captures fast, cheap gains but lacks the degrees of freedom to fully fix the policy's failure modes. **CEM-tiled** starts out slow in both the cases but ultimately does catch up the full-CEM performance, indicating a smaller subspace and a harder search problem upon enforcing this temporal constraint.

D.6.4 Effect of Sequential Halving on CEM

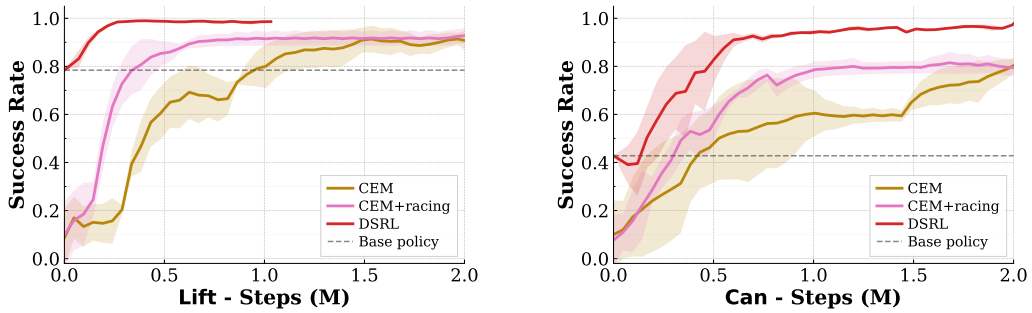
We next evaluate the effect of adding Sequential Halving [21] (*racing*) to CEM. Plain CEM evaluates every candidate ticket in the population on a fixed number of search envs per iteration (e.g. 20). CEM+racing uses the same population per iteration, but evaluates candidates in tiers: every candidate is first scored on a small number of envs (e.g. 5), the bottom half is then discarded, and only the survivors advance to the next tier where they accumulate 5 additional envs each (top half kept again, etc.). Surviving tickets eventually accumulate up to `base_per_tier` $\times$ `n_tiers` env evaluations (e.g. 25), while clearly losing tickets are eliminated after only 5 envs. Because most candidates never reach the later, more expensive tiers, the total number of env evaluations spent per CEM iteration is reduced compared to plain CEM at the same population size—this is the source of racing's sample-efficiency gain. To put these two CEM variants on a common axis, we also include DSRL [5] as a reference for what a strong gradient-based noise-policy method achieves under the same step budget. All three methods optimize over the same base policy and are measured by the same held-out success rate.

Within the CEM family, adding sequential halving substantially improves sample efficiency over plain CEM on both tasks while converging to a similar performance with enough iterations. The racing gain is therefore a sample-efficiency win from reallocating rollouts away from clearly losing candidates. The CEM+racing curve shows that this lightweight, no-training recipe still captures a large fraction of the achievable gain at a small fraction of the engineering and compute cost, and that there is potential to further improve the sample efficiency of searching for golden tickets with more sophisticated bandit algorithms.

D.7 Comparison with DSRL

D.7.1 Wall-clock Time Comparison

Searching for golden tickets is dramatically cheaper than DSRL in wall-clock time to reach a fixed env-step budget. Table 10 reports the wall-clock time taken by Plain CEM, CEM+racing, and DSRL to run 1M environment steps on `Lift` and 2M environment steps on `Can`, measured across three random seeds per method on similar hardware (single 24GB RTX3090/A5000 GPU with 24 core



(a) `Lift` (b) `Can`

Figure 13: Held-out success rate versus environment steps for plain CEM (gold), CEM with Sequential Halving (CEM+racing, pink), and DSRL (red) on robomimic `Lift` (left) and `Can` (right), with the base policy's success rate shown as a dashed line. Shaded regions show ± 1 standard deviation across 3 seeds.

CPU). Plain CEM completes the same env-step budget about $15\times$ faster than DSRL on `Lift` and `Can`, while CEM+racing is about $5\times$ faster than DSRL on both tasks. The CEM-over-DSRL gap reflects DSRL's additional cost of training an actor-critic noise-policy network on top of the rollouts, which CEM avoids entirely. **Importantly, DSRL's wall-clock times exacerbate with higher number of critics required to quell over-estimation (often 5-10) [5], as also used by our implementation for DexMimicGen results in Section D.7.2. This significantly increases the wall-clock time, often stretching into multiple days for complex visuomotor tasks**. In comparison, our method does not suffer from such issues for visual domains, as it is independent of the observations entirely, and scales purely with environment rollout times. Moreover, the entire search budget over tickets for random search can be parallelized disregarding hardware limitations. CEM and CEM-racing enforce more constraints in terms of sequential nature of their algorithms, increasing the wall-clock times (but also the sample-efficiency), while still being significantly faster than DSRL and not requiring any gradient updates. This reflects a similar parallelism gain obtained for gradient-free policy search methods such as evolution strategies [43], while being less sample efficient than gradient-based ones.

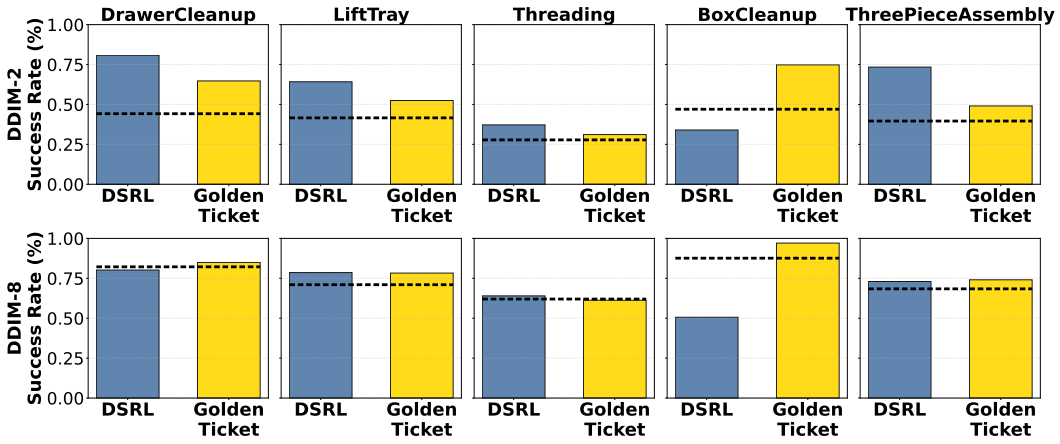
Table 10: Wall-clock time (minutes, mean over 3 random seeds) to run `Lift` for 1M environment steps and `Can` for 2M environment steps, comparing Plain CEM, CEM+racing, and DSRL on similar hardware. The bottom row reports the speed-up computed from the means.

Method	`Lift` (1M steps)	`Can` (2M steps)
Plain CEM	22.7	41.6
CEM+racing	66.8	149.1
DSRL	344.9 ($\approx$ 5.7 h)	827.1 ($\approx$ 13.8 h)
Speed-up vs. DSRL	$15.2\times$ / $5.2\times$ (CEM / racing)	$19.9\times$ / $5.5\times$ (CEM / racing)

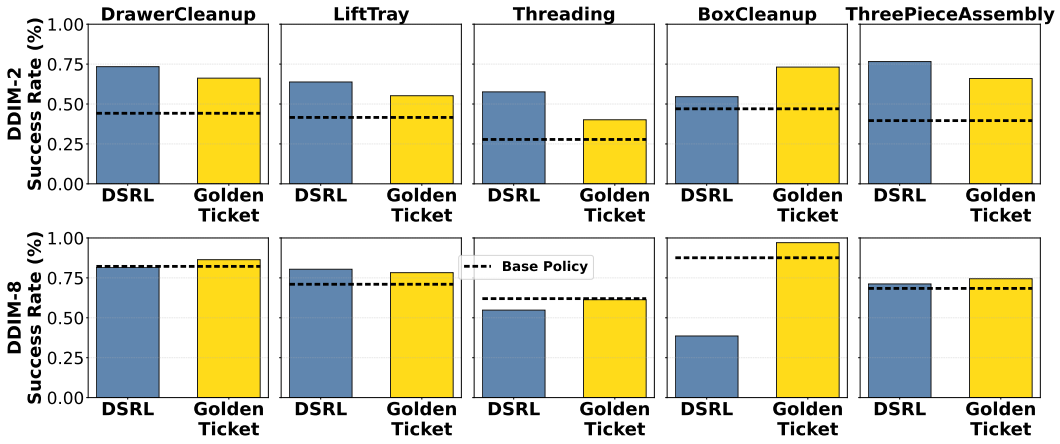
Setup. All three methods share the same frozen DPPO [3] MLP base policy for `Lift` and `Can`, with `ddim_steps` $= 8$, `act_steps` $= 4$, `action_dim` $= 7$, and `max_episode_steps` $= 300$. **Plain CEM** uses `n_envs` $= 20$, `n_eval_envs` $= 50$, `pop` $= 50$, `n_iter` $= 16$, `elite_frac` $= 0.2$, `init_std` $= 1.0$. **CEM+racing** uses a `tier5x5` schedule (`base_per_tier` $= 5$, `n_tiers` $= 5$, max 25 envs/ticket via successive halving), `pop` $= 50$, `n_iter` $= 33$, `elite_frac` $= 0.1$, $\alpha = 1.0$, `min_std` $= 0.1$, `extra_std` $= 0.1$. **DSRL** uses `dsrl_sac` with `n_envs` $= 4$, `n_eval_envs` $= 25$, `utd` $= 20$, batch size 256, `lr` $= 3\times10^{-4}$, $\gamma = 0.99$, $\tau = 0.005$, two critics with min backup, 3-layer 2048-hidden LayerNorm critic/actor MLPs, 20M-entry replay buffer, `init_rollout_steps` $= 1501$, and action clip ± 1.5 (as specified in the original DSRL codebase).

D.7.2 DexMimicGen Results

We elaborate on the DSRL [5] vs. golden-ticket comparison on DexMimicGen that we summarize in the main paper (Figure 4a). Figure 14 reports per-task success rate for DSRL and Golden Ticket (searched via Random Search) at two episode budgets (5k and 10k search episodes per task) crossed with two DDIM step counts (2 and 8); the dashed line in each panel marks the corresponding base-policy success rate. The five tasks are the standard DexMimicGen suite: `DrawerCleanup`, `LiftTray`, `Threading`, `BoxCleanup`, and `ThreePieceAssembly`. Golden tickets appear broadly equivalent to DSRL in sample efficiency on DexMimicGen. DSRL often struggles to match base-policy performance at DDIM-8, while doing better than golden tickets at DDIM-2. The standout task is `BoxCleanup`, where golden tickets clearly outperform DSRL across DDIM steps.



(a) 5k search episodes per task.



(b) 10k search episodes per task.

Figure 14: DSRL (**blue**) vs. Golden Ticket via Random Search (**yellow**) on DexMimicGen across five tasks (columns), two DDIM step settings (top row of each panel: DDIM-2; bottom row: DDIM-8), and two search-episode budgets (5k in (a), 10k in (b)). The dashed horizontal line in each panel marks the base-policy success rate at that DDIM setting.

D.8 Multi-task Results

D.8.1 LIBERO: per-task tickets that transfer across the suite.

Although each ticket in Table 1 was searched for a *single* task, several individual tickets match or improve the base policy on multiple tasks within the same suite. In each LIBERO suite, just 3 such tickets suffice to cover all 10 tasks (base → ticket per task):

- LIBERO-GOAL: `#03c2` (T0 72→84, T1 94→100, T2 86→92, T4 92→100, T6 80→94, T8 94→98); `#4d97` (T3 52→68, T5 78→92); `#672f` (T9 68→90).
- LIBERO-OBJECT: `#015a` (T2 98→100, T3 98→100, T5 82→92, T7 90→94, T8 96→100); `#184f` (T0 82→98, T4 76→100); `#14e7` (T1 98→100).
- LIBERO-SPATIAL: `#0e0c` (T0 78→80, T2 84→86, T3 62→100, T7 90→92, T8 78→84); `#6a21` (T1 94→98, T4 84→92); `#7c0e` (T5 58→60, T9 86→92).

D.8.2 SIMPLERENV (WidowX): multi-task golden tickets via joint search.

We complement the per-task SIMPLERENV results in Section D.4 with multi-task joint searches on the GR00T N1.5 base policy. Each search uses CEM with population 20 for up to 25 iterations and the task-averaged search-environment return as the objective; the top-ranked tickets returned by CEM are then re-evaluated on 300 held-out episodes per task. We run two families of searches:

- **2-task focused search** (Table 12), run separately on a drawer pair (`open_drawer`, `close_drawer`) and an eggplant pair (`put_eggplant_in_sink`, `put_eggplant_in_basket`), each with 10 instances per task (20 envs total).
- **7-task joint search** (Table 11) over all 7 SIMPLERENV WidowX tasks, in three configurations that differ in the number of search-environment instances allocated per task: 2/task (14 envs total), 5/task (35 envs total), and a heterogeneous *variable* allocation (10 each for `OD`, `COP`, `SOT`; 3 each for `CD`, `PEB`, `PES`, `SC`; 42 envs total).

For both cases, we search for a maximum of 500 tickets using CEM, except for the variable search case where we run it for up to 1000 tickets. Ticket#1 from the 2/task 7-task search lifts the task-averaged held-out success rate from 63.0 (base) to 77.4, matching the main paper's headline $+14\%$ improvement over 7 tasks. Ticket#1 from the eggplant 2-task search lifts the 2-task average from 42.0 (base) to 74.5, matching the main paper's $+30\%$ improvement on the eggplant pair.

As expected, we observe lower regression to the mean when the number of evaluations per ticket is raised. Interestingly, when optimizing over multiple tasks, the tasks with relatively higher base success rates (`OD`, `SOT`, `COP`; base 84, 82, 72) tend to be deprioritized by the search, as the attainable gains on them are smaller than on tasks with lower base policy performance. This may require an adjustment in the "weightage" assigned to each task in terms of the number of policy evaluations per ticket. We show this with the variable-allocation search in Table 11 (last two rows), which uses 10/task for the high-base tasks (`OD`, `COP`, `SOT`) and 3/task for the lower-base tasks (`CD`, `PEB`, `PES`, `SC`). Ticket#1 from this variable-allocation search improves the average held-out performance over the base policy by $+3.0\%$, and also improves on the high-performing tasks `COP` ($72 \rightarrow 82.7$) and `SOT` ($82 \rightarrow 83.3$), where the uniform 2/task and 5/task searches all regress. Golden tickets that outperform the base policy on a set of tasks by a relatively larger margin often drastically fail on other tasks, as noted before in our LIBERO results. However, we hypothesize that increasing the policy evaluation budget for each task to avoid regression to the mean, and adding per-task improvement constraints during search, should yield a ticket with a milder average improvement but no regression on any task. We leave this for future work.

Table 11: SIMPLERENV WidowX 7-task multi-task golden ticket search. Each row is a ticket returned by one of three multi-task CEM searches that differ in the number of search-environment instances allocated per task. **Search idx** is the chronological index of the ticket within its search (the candidate count at which it was first proposed). Cells are per-task held-out success rates (%) on 300 held-out episodes per task. **Avg.** is the task-averaged held-out success rate across the 7 tasks. **Search SR** is the ticket's task-averaged success rate inside the search environments (%). Task abbreviations: `CD=close_drawer`, `PEB=put_eggplant_in_basket`, `PES=put_eggplant_in_sink`, `SC=stack_cube`, `COP=carrot_on_plate`, `SOT=spoon_on_towel`, `OD=open_drawer`.

Search envs/task	Ticket	Search idx	CD	PEB	PES	SC	COP	SOT	OD	Avg.	Search SR
—	Base policy	—	65	63	21	54	72	82	84	63.0	—
2/task	ticket#1	367	85.7	**90.7**	**100.0**	58.7	55.0	67.3	84.7	**77.4**	100.0
5/task	ticket#1	319	91.0	88.3	64.0	71.3	57.7	70.0	84.7	75.3	94.3
	ticket#2	249	**96.7**	83.3	60.3	**77.0**	55.7	72.7	**93.7**	77.0	91.4
10/task: `OD,COP,SOT`	ticket#1	741	86.7	81.7	0.0	45.0	**82.7**	**83.3**	83.0	66.0	81.0
3/task: `CD,PEB,PES,SC`	ticket#2	771	92.7	71.0	47.0	50.3	80.3	78.0	83.3	71.8	81.0

Table 12: SIMPLERENV WidowX 2-task multi-task golden ticket search, for two task pairs run as separate searches: a drawer pair (`open_drawer`, `close_drawer`) and an eggplant pair (`put_eggplant_in_sink`, `put_eggplant_in_basket`). Both searches used 10 search-environment instances per task (20 envs total). **Search idx** is the chronological index of the ticket within its search. Cells are per-task held-out success rates (%) on 300 held-out episodes per task; **Avg.** is the 2-task average; **Search SR** is the ticket's task-averaged success rate inside the search environments (%).

Task pair	Ticket	Search idx	Task 1	Task 2	Avg.	Search SR
Drawer: `OD`, `CD`	Base policy	—	84	65	74.5	—
	ticket#1	201	90.0	**98.3**	94.2	100.0
	ticket#2	225	**97.3**	**98.3**	**97.8**	100.0
Eggplant: `PES`, `PEB`	Base policy	—	21	63	42.0	—
	ticket#1	382	60.7	**88.3**	**74.5**	100.0
	ticket#2	410	**64.0**	82.3	73.2	100.0